

Data Visualization

Edoardo Cortolezzis, Hari Prasad, Prathik Matthai

April 12, 2025

Contents

1	Introduction	4
2	Step 1	5
2.1	Introduction	5
2.2	Implementation & Results	5
2.2.1	Extracting nodes and edges	5
2.2.2	Random Visualization	5
2.2.3	Unique Random Positions	7
2.2.4	Sorted Circular Layout	8
2.3	Conclusions	9
3	Step 2	10
3.1	Introduction	10
3.2	BFS and DFS implementation	10
3.3	Layered layout	11
3.4	Results and Quality assessment	14
3.4.1	Time Complexity	14
3.4.2	BFS and DFS of Layered Tree Les Miserable	14
3.4.3	BFS and DFS Layered Trees with non-tree edges	15
3.4.4	DFS Layered Trees on League Network with Different Root Nodes	15
3.4.5	DFS Layered Trees on Les Miserables with Different Root Nodes	16
3.4.6	Other possible quality metrics for improvement	16
4	Step 3	17
4.1	Aim	17
4.2	Implementation	17
4.3	Result and Quality Assessment	19
4.3.1	Time Complexity	19
4.3.2	Parameters Choice	19
4.3.3	Jazz Network	19
4.3.4	Les Miserable	20
5	Step 4	21
5.1	Aim	21
5.2	Implementation	21
5.2.1	Time Complexity Analysis	21
5.2.2	Resolve cycle using Trivial Heuristic	21
5.2.3	Layer Assignment	24
5.2.4	Crossing minimization	27
5.2.5	Node Positioning	30
5.2.6	Edge Reversing and Drawing	33
5.3	Result and Quality Assessment	34
5.3.1	Results of the Sugiyama Framework	34
5.3.2	Crossing minimization	35
5.4	Bigger Graphs	35
6	Step 5	37
6.1	Aim	37
6.2	Implementation	37
6.2.1	Loading input graphs and their subgraphs	37
6.2.2	Applying Force-Directed Layout to Subgraphs within Bounding Boxes	37

6.2.3	Applying Force Directed Edge Bundling	37
6.2.4	Applying Smoothing	38
6.3	Results and Quality Assesment	39
7	Step 6	41
7.1	Aim	41
7.2	Implementation	41
7.2.1	Flyod-Warshall Algorithm and Distance Matrix Visualization	41
7.2.2	MDS	41
7.2.3	t-SNE	41
7.2.4	ISOMAP	42
7.3	Results and Quality Assessment	42
7.3.1	Distance Matrix	42
7.3.2	MDS	43
7.3.3	t-SNE	43
7.3.4	ISOMAP	44
8	Step 7	46
8.1	Aim	46
8.2	Quality metrics for Graph Visulaizations	46
8.2.1	Number of crossing	46
8.2.2	Crossing Resolution	46
8.2.3	Stresslayout	49
8.3	Quality metrics for the Projections for graphs	49
8.3.1	Trustworthiness and Continuity	49
8.3.2	Normalized Stress	52
8.3.3	Shepard Diagram	52
8.4	Results and Discussion	54
8.5	Force Directed Layouts	54
8.5.1	Layered Layout Graph	55
8.6	Projections for graph	57
8.7	Our favorite visualization technique	58
9	Discussion	59
9.1	About the Project	59
9.2	Code	59
10	Refrences	60

1 Introduction

For this project report, we worked on various data visualization techniques and worked on multiple datasets through which we generated visualizations. We begin with Step 1, implementing three fundamental layout algorithms—Random Visualization, Unique Random Positions, and Sorted Circular Layout—to establish baseline visual representations. Step 2 introduces hierarchical layouts using Breadth-First Search (BFS) and Depth-First Search (DFS), highlighting how traversal methods influence structural clarity. In Step 3, we apply the Fruchterman-Reingold algorithm, a force-directed approach, to optimize node placement and reveal natural clusters.

Step 4 focuses on the Sugiyama Framework, a method for visualizing directed graphs by resolving cycles, minimizing edge crossings, and improving readability. Step 5 extends force-directed principles to subgraphs, incorporating edge bundling and smoothing to enhance visual coherence. Step 6 explores dimensionality reduction techniques—MDS, t-SNE, and ISOMAP—to project high-dimensional graph data into 2D/3D space while preserving structural relationships. Finally, Step 7 evaluates visualization quality using metrics like edge crossings, stress, and neighborhood preservation.

Through these steps, we demonstrate how different algorithms and parameters impact graph readability, structural clarity, and computational efficiency. Our findings provide insights into selecting the best visualization techniques for various network types, balancing aesthetics, interpretability, and performance.

2 Step 1

2.1 Introduction

In the first step develop three layout algorithms for two separate datasets Les Misérables and the Jazz Network. The three algorithms Random Visualization, Unique Random Positions, Sorted Circular Layout are explained in further sections.

2.2 Implementation & Results

2.2.1 Extracting nodes and edges

The algorithm first reads a graph from the DOT file and extracts all nodes. It then iterates through each node to get their names and stores them in a list. Similarly, it extracts all edges from the graph and iterates through them to get the source and destination nodes for each edge, storing these as tuples in a separate list. Finally, it returns both the list of node names and the list of edges.

Algorithm 1 Key Components of DOT File Processing

```
1:  $G \leftarrow \text{read\_graph\_from\_dot\_file}(\text{data\_name}.1)$ 
2:  $\text{graph} \leftarrow G[0]$ 
3:  $\text{nodes} \leftarrow \text{get\_nodes}(\text{graph})$ 
4: for each node in nodes do
5:    $\text{node\_name} \leftarrow \text{get\_name}(\text{node})$ 
6:    $\text{append}(\text{node\_names}, \text{node\_name})$ 
7: end for
8:  $\text{edges} \leftarrow \text{get\_edges}(\text{graph})$ 
9: for each edge in edges do
10:   $\text{append}(\text{edge\_list}, (\text{get\_source}(\text{edge}), \text{get\_destination}(\text{edge})))$ 
11: end for
```

2.2.2 Random Visualization

We retrieve all nodes and edges from the graph object and initialize an empty dictionary to store node positions. For each node, it generates random 2D coordinates, creates nodes with circles, and stores the position. For each edge, it retrieves the positions of the source and destination nodes and draws a connecting line between them. Finally, it sets the plot limits, removes the axes, and displays the graph. The dataset used here is JazzNetwork.

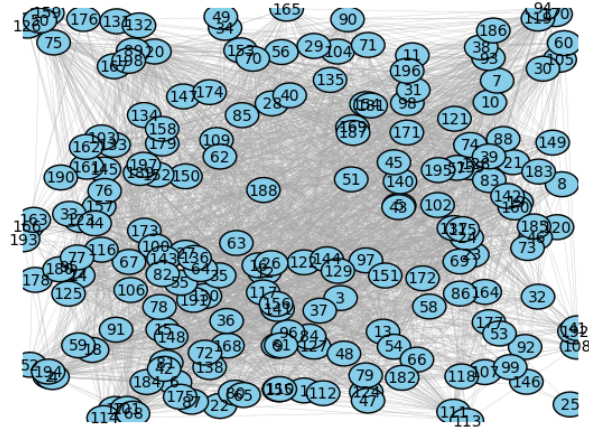


Figure 1: Random Visualization

Algorithm 2 Random Visualization

```

1:  $nodes \leftarrow \text{graph.get\_nodes}()$ 
2:  $edges \leftarrow \text{graph.get\_edges}()$ 
3:  $node\_positions \leftarrow \{\}$ 
4: for all  $node \in nodes$  do
5:    $pos \leftarrow \text{random point in } [0, 1]^2$ 
6:    $node\_positions[node.get\_name()] \leftarrow pos$ 
7: end for
8: for all  $edge \in edges$  do
9:    $source\_pos \leftarrow node\_positions[edge.get\_source()]$ 
10:   $dest\_pos \leftarrow node\_positions[edge.get\_destination()]$ 
11:  Draw dotted line from  $source\_pos$  to  $dest\_pos$  using color #A9A9A9
12: end for
13: for all  $node \in nodes$  do
14:    $pos \leftarrow node\_positions[node.get\_name()]$ 
15:   Draw circle at  $pos$  with radius 0.03, blue fill and black edge
16:   Place label  $node.get\_name()$  centered at  $pos$ 
17: end for
18: Display the figure

```

2.2.3 Unique Random Positions

This algorithm differs from the previous one in that each node is assigned a unique random position with a minimum Euclidean distance of 0.5 compared to others. To prevent placement at the canvas edges, positions are restricted to the $[1, 9]$ interval in each dimension. Finally, to limit computation time for larger graphs, the algorithm attempts at most 100 times to find a unique position for each node. Figure 2 represents the LesMiserables data set, however, for the JazzNetwork dataset, some issues arose where a few nodes were being placed very close to each other near at (0,0). The time complexity is $O(n^2)$.

Algorithm 3 Unique Random Positions

```
1: function UNIQUE_RANDOM_POSITIONS(existing_positions, max_attempts = 100)
2:   for  $i = 1$  to max_attempts do
3:     Generate random position pos in range  $(0.1, 0.9) \times 10$ 
4:     if  $\forall \text{existing} \in \text{existing\_positions}, \text{distance}(\text{pos}, \text{existing}) > 0.5$  then
5:       return pos
6:     end if
7:   end for
8:   return fallback random position
9: end function
10: Get list of nodes and edges from graph
11: Initialize empty node_positions
12: Setup plot
13: Define option to show/hide node and edge labels
14: for each node do
15:   if node name is numeric then
16:     Assign unique random position
17:     Draw node as circle
18:   if node labels enabled then
19:     Display node label at its position
20:   end if
21: end if
22: end for
23: for each edge do
24:   if source and target are numeric then
25:     Draw dashed line between source and target
26:   if edge labels enabled then
27:     Display weight at midpoint
28:   end if
29: end if
30: end for
31: Finalize plot display
```

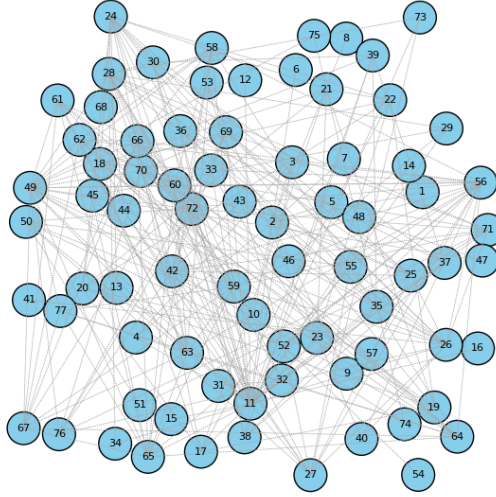


Figure 2: Unique Random Position

2.2.4 Sorted Circular Layout

For this visualization, we first convert the input graph to a NetworkX graph and calculate node degrees. Then, nodes are sorted by degree and arranged in a circular layout with the most connected node at 0 degrees, followed by less connected nodes in an anticlockwise order. Finally, nodes are connected with grey dotted edges. The time complexity here is $O(n \log(n))$.

Algorithm 4 Sorted Circular Layout

```

1:  $G_{nx} \leftarrow nx.nx\_pydot.from\_pydot(graph)$ 
2:  $node\_degrees \leftarrow dict(G_{nx}.degree())$ 
3:  $sorted\_nodes \leftarrow sorted(node\_degrees, key = node\_degrees.get, reverse = True)$ 
4:  $angles \leftarrow np.linspace(0, 2\pi, num\_nodes, endpoint = False)$ 
5:  $radius \leftarrow 5$ 
6: for  $i \leftarrow 0$  to  $num\_nodes - 1$  do
7:    $angle \leftarrow angles[i]$ 
8:    $x \leftarrow radius \cdot \cos(angle)$ 
9:    $y \leftarrow radius \cdot \sin(angle)$ 
10:   $node\_positions[sorted\_nodes[i]] \leftarrow (x, y)$ 
11: end for
12: for each  $node, pos$  in  $node\_positions$  do
13:    $ax.add\_patch(Circle(pos, radius = 0.1, edgecolor = 'black', facecolor = 'skyblue'))$ 
14: end for
15: for each  $edge$  in  $G_{nx}.edges()$  do
16:    $ax.add\_patch(ConnectionPatch(source\_pos, target\_pos, color = '#A9A9A9'))$ 
17: end for

```



Figure 3: Circular Layout

2.3 Conclusions

We created three network visualization methods—random layout, unique-position random layout, and sorted circular layout— tested with both the Les Misérables and Jazz Network datasets. The strengths and trade-offs of each algorithm are as follows.

Random Layout:

- *Pros:* Simple implementation and fastest execution.
- *Cons:* Arbitrary node placement leads to severe visual clutter and minimal insight into network structure.

Unique-Position Random Layout:

- *Pros:* Enforces a minimum Euclidean distance (0.5) between nodes, reducing overlap and improving readability.
- *Cons:* In the Jazz Network graph we had some issues due to some nodes plotted near (0,0) and overlapped, impacting visualization quality.

Random Layout:

- *Pros:* Sorts nodes by degree—starting with the most connected at 0°—to clearly reveal network hierarchy and connectivity.
- *Cons:* While structured, the circular arrangement may obscure natural groupings within the network.

Regarding quality metrics, none of the layouts fully minimize edge crossings. However, confining node placement to a restricted space (a square for the random methods and a circle for the sorted layout) partially mitigated clutter, by confining the nodes in a limited area. Overall, the sorted circular layout offered the best balance by emphasizing network structure, despite its potential to mask inherent groupings.

3 Step 2

3.1 Introduction

Our study implements layered tree layouts to visualize Breadth-First Search (BFS) and Depth-First Search (DFS) spanning trees across multiple network datasets, including Les Misérables, Jazz and League Network. We optimize memory usage and traversal efficiency through an adjacency list representation, which associates each node with its connected neighbors for constant-time access. To support layered visualization, we extend this foundation with a hierarchical structure that captures both parent-child relationships and depth information from traversals. This dual representation efficiently organizes nodes into vertical layers based on their distance from the root while preserving network connectivity information, allowing us to analyze how different traversal algorithms and root node selections affect structural representation.

3.2 BFS and DFS implementation

Algorithm 5 BFS Tree Construction

```
1: procedure BFS_TREE(graph, root)
2:   visited  $\leftarrow \emptyset$  ▷ Initialize empty set
3:   queue  $\leftarrow [root]$  ▷ Initialize queue with root
4:   tree  $\leftarrow \{root : []\}$  ▷ Initialize tree
5:   while queue is not empty do
6:     node  $\leftarrow$  dequeue from queue
7:     Add node to visited
8:     for each neighbor of node in graph do
9:       if neighbor  $\notin$  visited then
10:        Add neighbor to visited
11:        Enqueue neighbor to queue
12:        Append neighbor to tree[node]
13:        tree[neighbor]  $\leftarrow []$  ▷ Initialize empty children list
14:       end if
15:     end for
16:   end while
17:   return tree
```

Algorithm 6 DFS Tree Construction

```
1: procedure DFS_TREE(graph, root, visited = null, tree = null)
2:   if visited is null then
3:     visited  $\leftarrow \emptyset$  ▷ Initialize empty set
4:   end if
5:   if tree is null then
6:     tree  $\leftarrow \{\text{root} : []\}$  ▷ Initialize tree
7:   end if
8:   Add root to visited
9:   for each neighbor of root in graph do
10:    if neighbor  $\notin$  visited then
11:      Append neighbor to tree[root]
12:      tree[neighbor]  $\leftarrow []$  ▷ Initialize empty children list
13:      DFS_TREE(graph, neighbor, visited, tree) ▷ Recursive call
14:    end if
15:  end for
16:  return tree
17: end procedure = 0
```

3.3 Layered layout

The draw layered tree function visualizes hierarchical tree structures with an emphasis on proper node spacing and level organization. There are two phases, first, a postorder traversal computes optimal horizontal spacing between nodes to prevent overlaps between subtrees; second, a pre-order traversal assigns final coordinates based on depth and calculated spacing. The algorithm handles complex cases where nodes have multiple children by ensuring adequate separation and, if necessary, shifting entire subtrees to maintain layout integrity. This displays the hierarchical relationships in the tree while maintaining level boundaries, with horizontal dotted lines indicating different depths.

Algorithm 7 Layered Tree

```
1: function DRAWLAYEREDTREE(tree, title, show_node_ids)
2:   spacing  $\leftarrow \{\}$ 
3:   boundaries  $\leftarrow \{\}$ 
4:   order  $\leftarrow 0$ 
5:   for node  $\in$  tree do
6:     spacing[node]  $\leftarrow$  order
7:     order  $\leftarrow$  order + 2
8:   end for
9:   root  $\leftarrow$  next(iter(tree))
10:  COMPUTEPOSTORDERSPACING(tree, root, spacing, boundaries)
11:  x_positions  $\leftarrow \{\}$ 
12:  y_positions  $\leftarrow \{\}$ 
13:  ASSIGNPREORDERPOSITIONS(tree, root, x_positions, y_positions, 0, spacing)
14:  for parent  $\in$  tree do
15:    for child  $\in$  tree[parent] do
16:      Plotline      from      (x_positions[parent], y_positions[parent])      to
      (x_positions[child], y_positions[child])
17:    end for
18:  end for
19:  if show_node_ids then
20:    for node  $\in$  x_positions do
21:      Draw node label at (x_positions[node], y_positions[node])
22:    end for
23:  else
24:    for node  $\in$  x_positions do
25:      Draw empty node at (x_positions[node], y_positions[node])
26:    end for
27:  end if
28:  for level  $\in$  unique levels do
29:    Draw horizontal line at y = level
30:    Label level at (min(x_positions) - 15.5, level + 0.1)
31:  end for
32:  Display title and axis
33: end function
```

Algorithm 8 Compute Postorder Spacing

```
1: function COMPUTEPOSTORDERSPACING(tree, node, spacing, boundaries,  
   min_separation = 1.0)  
2:   if node  $\notin$  tree or tree[node] is empty then  
3:     boundaries[node]  $\leftarrow$  (spacing[node], spacing[node])  
4:     return  
5:   end if  
6:   children  $\leftarrow$  tree[node]  
7:   for child  $\in$  children do  
8:     COMPUTEPOSTORDERSPACING(tree, child, spacing, boundaries, min_separation)  
9:   end for  
10:  if len(children)  $\geq$  3 then  
11:    for i  $\leftarrow$  0 to len(children) - 2 do  
12:      current_child  $\leftarrow$  children[i]  
13:      next_child  $\leftarrow$  children[i + 1]  
14:      current_right  $\leftarrow$  boundaries[current_child][1]  
15:      next_left  $\leftarrow$  boundaries[next_child][0]  
16:      if next_left - current_right < min_separation then  
17:        shift_amount  $\leftarrow$  (current_right + min_separation) - next_left  
18:        SHIFTSUBTREE(tree, next_child, shift_amount, spacing, boundaries)  
19:      end if  
20:    end for  
21:  end if  
22:  leftmost  $\leftarrow$  min(boundaries[child][0] | child  $\in$  children)  
23:  rightmost  $\leftarrow$  max(boundaries[child][1] | child  $\in$  children)  
24:  midpoint  $\leftarrow$  (leftmost + rightmost)/2  
25:  spacing[node]  $\leftarrow$  midpoint  
26:  boundaries[node]  $\leftarrow$  (leftmost - padding, rightmost + padding)  
27: end function
```

Algorithm 9 Shift Subtree

```
1: function SHIFTSUBTREE(tree, node, shift_amount, spacing, boundaries)  
2:   spacing[node]  $\leftarrow$  spacing[node] + shift_amount  
3:   boundaries[node]  $\leftarrow$  (boundaries[node][0] + shift_amount, boundaries[node][1] +  
   shift_amount)  
4:   if node  $\in$  tree then  
5:     for child  $\in$  tree[node] do  
6:       SHIFTSUBTREE(tree, child, shift_amount, spacing, boundaries)  
7:     end for  
8:   end if  
9: end function
```

Algorithm 10 Assign Preorder Positions

```
1: function ASSIGNPREORDERPOSITIONS(tree, node, x_positions, y_positions, depth,  
   spacing)  
2:   x_positions[node]  $\leftarrow$  spacing[node]  
3:   y_positions[node]  $\leftarrow$   $-depth$   
4:   for child  $\in$  tree[node] do  
5:     ASSIGNPREORDERPOSITIONS(tree, child, x_positions, y_positions, depth + 1,  
       spacing)  
6:   end for  
7: end function
```

3.4 Results and Quality assessment

3.4.1 Time Complexity

In the BFS tree function, each node is visited exactly once and each edge is examined at most once. The Queue operations (append and popleft) are $O(1)$ and Checking if a node is in a set (visited) is $O(1)$. So, the time complexity should be $O(V + E)$ where V is the number of vertices and E is the number of edges in the graph.

In the DFS tree function each node is visited exactly once through the recursive call and edges are examined at most once. Set operations (adding to visited) are $O(1)$ and the recursive call stack can be at most V deep. Hence, the resulting time complexity is $O(V + E)$. Where V is the number of vertices and E is the number of edges in the graph.

3.4.2 BFS and DFS of Layered Tree Les Miserable

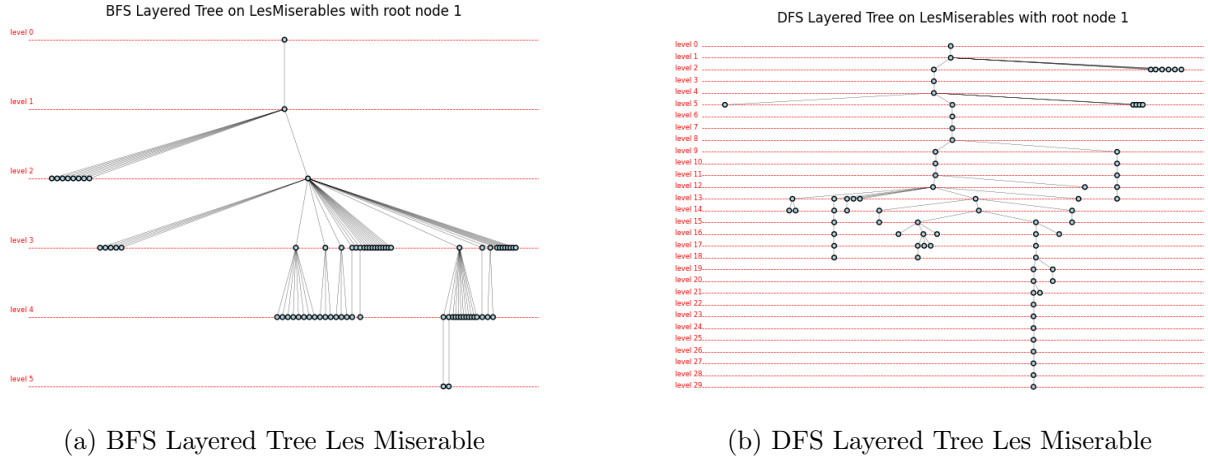


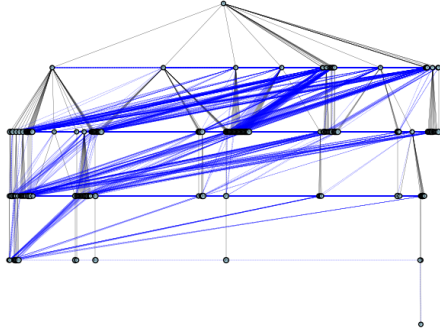
Figure 4: Comparison of BFS and DFS Layered Trees on the Les Miserables Dataset

Looking at the two visualizations of the Les Miserables network, the BFS (Breadth-First Search) tree creates a much more balanced and visually informative representation compared to the DFS (Depth-First Search) tree. The BFS visualization efficiently clusters nodes at the same depth, resulting in a compact, wide tree with a clear hierarchical structure spanning only 5 levels. In contrast, the DFS visualization produces an elongated, thinner tree that extends up to 29 levels, often forming long vertical chains with single nodes per level.

The layered layout avoids any edge crossings or bends in the tree edges, and although it doesn't minimize the overall drawn area, it offers a good intuition of the graph's structure. Overall, the wider and shorter structure of the BFS layout provides a more interpretable view of the kinship relationships of the nodes compared to the DFS layout.

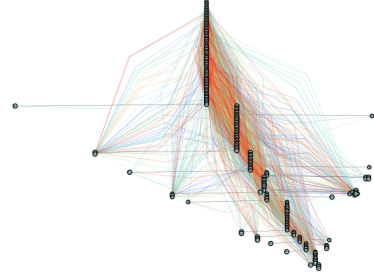
3.4.3 BFS and DFS Layered Trees with non-tree edges

BFS Layered Tree on JazzNetwork with root node 1 and straight blue non tree edges



(a) BFS Layered Tree Jazz Network

DFS Layered Tree on JazzNetwork with root node 1 and randomly colored and bended non tree edges



(b) DFS Layered Tree Jazz Network

Figure 5: Comparison of BFS and DFS Layered Trees on the Jazz Network Dataset

We experimented with two methods for displaying non-tree edges. The first approach used blue dashed lines, but when multiple edges appeared on the same level, they overlapped and made it hard to see which nodes were connected. To improve clarity, we then applied colored bent edges to highlight each connection. However, due to the layered layout and the randomness in edge coloring and bending, the edges still sometimes clustered and appeared chaotic, making the overall visualization slightly difficult to interpret.

3.4.4 DFS Layered Trees on League Network with Different Root Nodes

../Outputs/LeagueNetwork/graph_LeagueNetwork-dfs_on_node_1.png



(a) DFS Layered Tree League Network (root node = 1)

../Outputs/LeagueNetwork/graph_LeagueNetwork-dfs_on_node_15.png



(b) DFS Layered Tree League Network (root node = 15)

Figure 6: Comparison of DFS Layered Trees on the League Network Dataset with Different Root Nodes

Looking at the two DFS visualizations of the League Network, both images display nearly identical linear chain structures despite having different root nodes (node 15 in the first image and node 1 in the second). The visualizations organize all 16 nodes into a straight vertical sequence with no branching, forming a simple path where each node has exactly one child (except for the leaf node at the bottom). The node order differs between the two visualizations based on the starting point, with the root node appearing at the top and subsequent nodes arranged in the order they were discovered during the DFS traversal. This pattern suggests the League Network likely has a specific underlying topology - possibly a simple cycle, path, or chain-like structure -

that constrains the DFS algorithm to discover nodes sequentially. Both visualizations maintain the same set of nodes (numbered 1-16) but arrange them in different sequences based on the traversal order determined by the respective root nodes. The structural similarity between both trees indicates that the network topology produces consistent DFS traversal patterns regardless of the starting vertex.

3.4.5 DFS Layered Trees on Les Miserables with Different Root Nodes

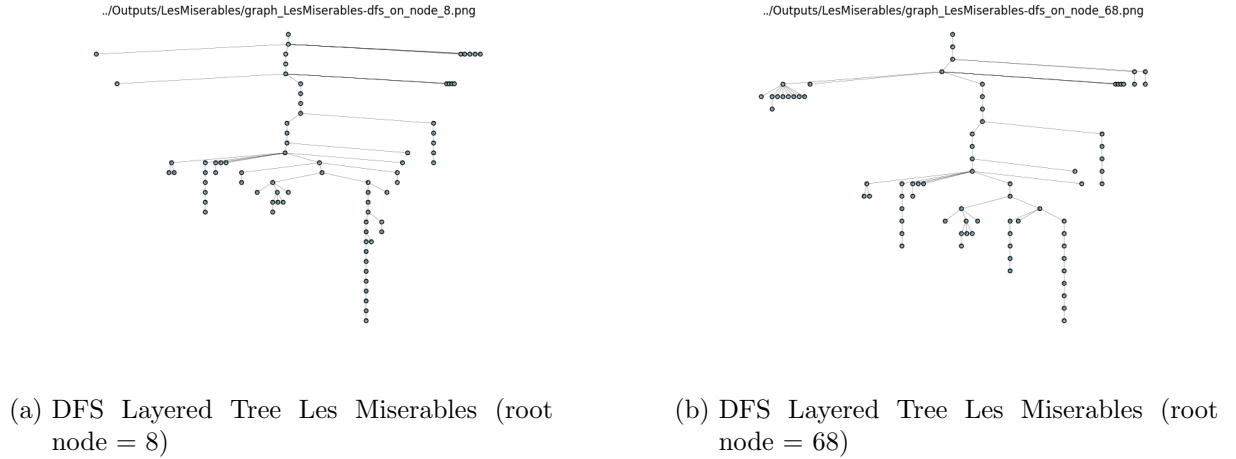


Figure 7: Comparison of DFS Layered Trees on the Les Miserables Dataset with Different Root Nodes

Comparing these two DFS visualizations of the Les Miserables network reveals structural differences stemming from the choice of root nodes. The tree rooted at node 68 displays a central vertical path with several horizontal branches extending outward, creating clusters of nodes at different depths. It features multiple small groupings of nodes distributed asymmetrically throughout the structure, with some notable dense clusters in the left and right upper regions. The tree rooted at node 8 maintains a similar overall silhouette but organizes nodes differently within that framework, it has a somewhat evenly distributed branching pattern along its main vertical path, with slightly different node clustering arrangements, particularly in the middle sections of the visualization. Both trees share a characteristic vertical orientation with a central backbone structure from which horizontal branches extend, though the specific pattern of these branches varies between the two visualizations. The distribution of leaf nodes and the depth of various branches reflect the different traversal paths determined by the starting points (node 68 versus node 8), resulting in distinct node orderings and subtree arrangements while maintaining the fundamental DFS traversal pattern.

3.4.6 Other possible quality metrics for improvement

To further improve our visualization, additional quality metrics can be considered. First, enhancing readability would allow the visualization to be understood better. Second, the distribution of the nodes could be improved in these visualizations by implementing more uniform spacing between nodes at each level and reducing the overcrowding visible in certain areas. Finally Level assignment in the visualizations could be optimized by implementing a more even distribution of nodes across fewer levels, particularly in the DFS layouts where numerous sparse levels create unnecessary vertical stretching

4 Step 3

4.1 Aim

In this section, we will be implementing a force directed layout using the Fruchterman-Reingold algorithm [1], along with the discussions on the choices of our forces/parameters. The results display the output from the Les Misérables network and Jazz network data sets, followed by a discussion on the results and quality.

4.2 Implementation

We chose to implement the Fruchterman-Reingold variant for visualizing force-directed graphs. This variant was chosen primarily due to its simplicity, as it requires only one primary parameter to adjust, in addition to the number of iterations and the minimum movement threshold. In contrast, the Eades algorithm offers more parameters that can be fine-tuned, leading to a wider variety of visualization outcomes. However, in order to test different values of the parameter, the Fruchterman-Reingold algorithm was preferred due to its simplicity in the parameter choice.

Moreover, we incorporated node coloring to visually represent the level of connectivity between them. The range of different colors allow clusters of densely connected nodes to be easily identified. Furthermore this coloring denotes how nodes with lower connectivity are pushed towards the periphery of the canvas. To further enhance the algorithm, we added inertia by considering node degree when calculating attractive forces, making highly-connected nodes more stable and closely grouped. We also implemented boundary constraints to prevent nodes from being pushed outside the canvas—after calculating each node’s new position, we check if it falls outside the canvas boundaries and, if so, reposition it to the nearest point within the boundaries with a small margin. These enhancements provide a clearer understanding of the graph’s structure and the relationships between its nodes.

Algorithm 11 Drawing Colored Force-Directed Graph (Fruchterman-Reingold with Inertia)

```
1: function COLORFULFORCEDIRECTEDGRAPH(file_path, iterations, C, save_frames, output_dir,  
   threshold, display_ids)  
2:   Create output directory if needed  
3:   Load graph from DOT file and extract nodes and edges from graph  
4:   Compute node_degrees for each node and use it to create a colormap with range  
   [min_degree, max_degree]  
5:   Calculate node_masses  $\leftarrow \{node : 1 + node\_degrees[node]/2 \mid node \in nodes\}$   $\triangleright$  For  
   inertia  
6:   Scale the drawings accordingly with the number of nodes (n)  
7:   Calculate the ideal edge length:  $l \leftarrow C \times \sqrt{area/n}$   $\triangleright$  Optimal edge length  
8:   Assign initial random positions pos to all nodes  
9:   max_movement  $\leftarrow \infty$   
10:  t  $\leftarrow 0.1$   $\triangleright$  Initialize the temperature that will decrease with the following iterations  
11:  i  $\leftarrow 0$   
12:  while i < iterations and max_movement > threshold do  
13:    Store current positions in old_positions  
14:    Initialize force vectors: f_rep, f_attr, f_total  
15:    for each pair of nodes (u, v) where u  $\neq$  v do  
16:      delta  $\leftarrow pos[v] - pos[u]$   
17:      distance  $\leftarrow \max(0.1, \|delta\|)$   
18:      repulsive_magnitude  $\leftarrow l^2/distance$   
19:      direction  $\leftarrow delta/distance$   
20:      f_rep[u]  $\leftarrow f\_rep[u] - direction \times repulsive\_magnitude$   
21:    end for  
22:    for each edge (u, v)  $\in edges$  do  
23:      delta  $\leftarrow pos[v] - pos[u]$   
24:      distance  $\leftarrow \max(0.1, \|delta\|)$   
25:      attractive_magnitude  $\leftarrow distance^2/l$   
26:      direction  $\leftarrow delta/distance$   
27:      f_attr[u]  $\leftarrow f\_attr[u] + direction \times (attractive\_magnitude/node\_masses[u])$   $\triangleright$   
      Inertia  
28:    end for  
29:    for each node v  $\in nodes$  do  
30:      f_total[v]  $\leftarrow f\_rep[v] + f\_attr[v]$   
31:      pos[v]  $\leftarrow pos[v] + t \times f\_total[v]$   $\triangleright$  Update position based on total force  
32:      pos[v][0]  $\leftarrow \min(width - margin, \max(margin, pos[v][0]))$   $\triangleright$  Constrain  
      x-coordinate  
33:      pos[v][1]  $\leftarrow \min(height - margin, \max(margin, pos[v][1]))$   $\triangleright$  Constrain  
      y-coordinate  
34:    end for  
35:    Calculate max_movement across all nodes  
36:    t  $\leftarrow t \times 0.95$   $\triangleright$  Decreasing the impact of following forces  
37:    if i mod 10 = 0 or i = iterations - 1 or max_movement  $\leq threshold$  then  
38:      Draw edges between connected nodes  
39:      Draw nodes with colors based on degree  
40:      if display_ids then  
41:        Display node IDs  
42:      end if  
43:      Add colorbar showing degree scale  
44:      if save_frames then  
45:        Save current visualization frame  
46:      end if  
47:    end if  
48:    i  $\leftarrow i + 1$   
49:  end while  
50:
```

4.3 Result and Quality Assessment

4.3.1 Time Complexity

The time complexity of this algorithm is dominated by the nested loops computing repulsive and attractive forces. The outer loop runs for *iterations*, and within each iteration, repulsive forces are calculated between all node pairs ($O(n^2)$), while attractive forces are computed along edges ($O(m)$). Thus, the overall complexity is $O(\text{iterations} \times n^2)$, as the repulsive force calculation dominates. In sparse graphs ($m \approx n$), this remains $O(\text{iterations} \times n^2)$, and in dense graphs ($m \approx n^2$), it stays the same. Other operations, such as cooling and drawing, contribute linear or constant factors, which are negligible.

4.3.2 Parameters Choice

Regarding the parameters, the convergence threshold value is set to 0.001, and the maximum number of iteration to be 300. The C value is set at a range of (0.1, 0.2, 0.75, and 1.5). The difference in the C values is discussed below

4.3.3 Jazz Network

The Jazz Network visualization 8 demonstrates the dramatic effect of increasing C-values in the Fruchterman-Reingold algorithm across four different parameterizations. In the leftmost panel (C=0.1, iteration 88), the network forms a compact, centralized structure with high-degree nodes (magenta) clustered in the center and lower-degree nodes (blue) radiating outward in a star-like pattern. This concentrated layout effectively reveals the hierarchical nature of the jazz network with a small maximum movement value (0.00997), indicating the layout is approaching stability.

As the C-value increases across the panels (C=0.2, C=0.75, and C=1.5), we observe a progressive expansion of the network structure. At C=0.2, iteration 85 shows the network beginning to spread while maintaining some centrality. At C=0.75, the network becomes significantly more distributed, with nodes filling more of the available space and community structures becoming more apparent. In C=1.5, we see the most dispersed layout, with nodes pushed toward the boundaries, clearly separating the clusters while still preserving connection patterns.

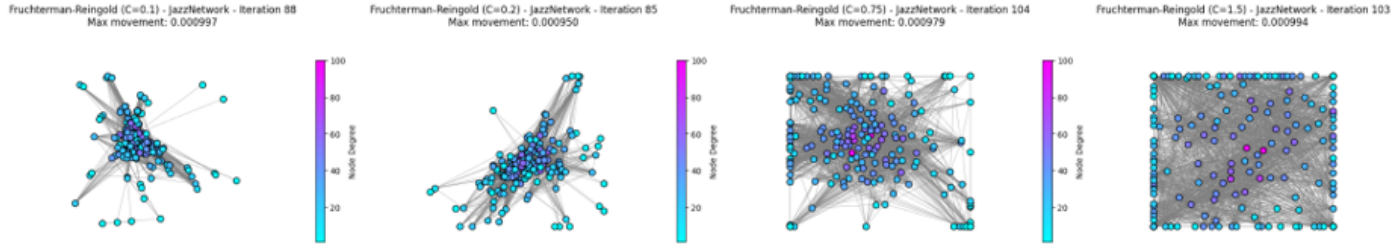


Figure 8: Jazz Network Layered Layout

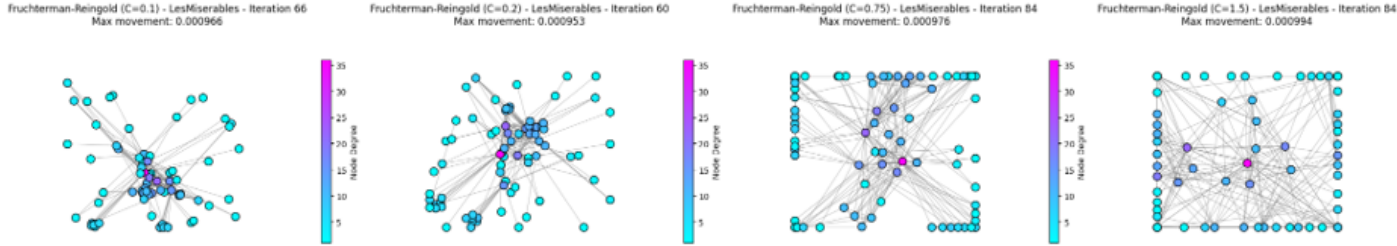


Figure 9: Les Miserable Layer Layout

4.3.4 Les Miserable

The Les Misérables network visualizations⁹ illustrate the significant impact of varying C-values in the Fruchterman-Reingold algorithm. At $C=0.1$, the network displays a compact structure where nodes with higher connections are clustered in the center, with nodes with fewer connections branching outward.

As the C-value increases through the panels ($C=0.2$, $C=0.75$, and $C=1.5$), the character network progressively expands and reveals more distinct community structures. At $C=0.2$, shows slight expansion while maintaining centrality. At $C=0.75$, the graph becomes more distributed, with clearer separation between different the nodes. At $C=1.5$, presents the most dispersed layout, arranging characters in a nearly square boundary that maximizes the distances between different character communities. from centralized interactions at low C values to distinct community structures at higher C values.

5 Step 4

5.1 Aim

This step computes the visualization of directed graph through the Sugiyama Framework. It starts with a given graph that may contain cycles, for which it reverses the edges that cause loops and proceed with displaying the nodes on a layered layout that minimizes the number of layers used and adds dummy nodes for edges that span more than 1 layer. Then the edge crosses get minimized and nodes are re-placed on the layers to straighten the edges in vertical lines. Finally, the reversed edges get reset to their original directions, the dummy nodes are removed and their edges get replaced by some Bezier curves.

5.2 Implementation

The different steps of the Sugiyama Framework is implemented and their pseudo code are as follows.

5.2.1 Time Complexity Analysis

Below is a detailed notions used for time complexity.

- n be the number of nodes,
- e be the number of edges,
- L be the number of layers (for layered functions),
- I be the number of iterations (when loops are based on a maximum iteration count), and
- Other symbols (like P for the number of paths, or p for the length of a path) when needed.

5.2.2 Resolve cycle using Trivial Heuristic

Algorithm 12 Trivial Heuristic for Cycle Removal

```
1: function TRIVIALHEURISTIC(nodes, edges)
2:   node_pos_ordered  $\leftarrow \emptyset$  ▷ Create node positions ordering
3:   for  $i = 0$  to  $|nodes| - 1$  do
4:     node_pos_ordered[nodes[ $i$ ]]  $\leftarrow i$ 
5:   end for
6:    $A \leftarrow \emptyset$  ▷ edges to keep (left to right)
7:    $A_{reverse} \leftarrow \emptyset$  ▷ edges to reverse (right to left)
8:   for each  $(src, dest) \in edges$  do
9:     if node_pos_ordered[dest] < node_pos_ordered[src] then
10:       $A_{reverse} \leftarrow A_{reverse} \cup \{(src, dest)\}$ 
11:     else
12:       $A \leftarrow A \cup \{(src, dest)\}$ 
13:     end if
14:   end for
15:   return ( $A, A_{reverse}$ )
16: end function
```

Overall time Complexity for both the algorithms is : $O(n + e)$.

Algorithm 13 Edge Reversal for Cycle Removal

[H]

```
1: function REVERSED_EDGES( $A, A_{reverse}$ )  
2:    $A_{reversed} \leftarrow \emptyset$   
3:   for each  $(src, dst) \in A_{reverse}$  do  
4:      $A_{reversed} \leftarrow A_{reversed} \cup \{(dst, src)\}$   
5:   end for  
6:    $final\_reversed \leftarrow A_{reversed} \cup A$   
7:   return ( $final\_reversed, A_{reversed}$ )  
8: end function
```

5.2.3 Layer Assignment

Algorithm 14 Sugiyama Layer Assignment with Dummy Nodes

```

1: function SUGIYAMALAYERASSIGNMENT(nodes, edges)
2:   all_nodes  $\leftarrow$  set of nodes
3:   for each (src, dst)  $\in$  edges do
4:     all_nodes  $\leftarrow$  all_nodes  $\cup$  {src, dst}
5:   end for
6:   sink_nodes  $\leftarrow$  FindSinks(nodes, edges)
7:   working_nodes  $\leftarrow$  all_nodes  $\setminus$  sink_nodes
8:   working_edges  $\leftarrow$  {(src, dst)  $\in$  edges | dst  $\notin$  sink_nodes}
9:   layers  $\leftarrow$   $\emptyset$ 
10:  remaining_nodes  $\leftarrow$  working_nodes
11:  remaining_edges  $\leftarrow$  working_edges
12:  while remaining_nodes  $\neq$   $\emptyset$  do
13:    sources  $\leftarrow$  FindSources(remaining_nodes, remaining_edges)
14:    if sources =  $\emptyset$  then
15:      raise error "Graph contains cycles or is disconnected."
16:    end if
17:    layers  $\leftarrow$  layers  $\cup$  {sources}
18:    remaining_nodes  $\leftarrow$  remaining_nodes  $\setminus$  sources
19:    remaining_edges  $\leftarrow$  {(src, dst)  $\in$  remaining_edges | src  $\notin$  sources}
20:  end while
21:  if sink_nodes  $\neq$   $\emptyset$  then
22:    layers  $\leftarrow$  layers  $\cup$  {sink_nodes}
23:  end if
24:  return AddDummyNodes(layers, edges)
25: end function
26: function FINDSOURCES(nodes, edges)
27:   all_nodes  $\leftarrow$  set of nodes
28:   for each (src, dst)  $\in$  edges do
29:     all_nodes  $\leftarrow$  all_nodes  $\cup$  {src, dst}
30:   end for
31:   incoming_edges  $\leftarrow$  map from each node  $\in$  all_nodes to 0
32:   for each (src, dst)  $\in$  edges do
33:     incoming_edges[dst]  $\leftarrow$  incoming_edges[dst] + 1
34:   end for
35:   return {node  $\in$  all_nodes | incoming_edges[node] = 0}
36: end function
37: function FINDSINKS(nodes, edges)
38:   all_nodes  $\leftarrow$  set of nodes
39:   for each (src, dst)  $\in$  edges do
40:     all_nodes  $\leftarrow$  all_nodes  $\cup$  {src, dst}
41:   end for
42:   outgoing_edges  $\leftarrow$  map from each node  $\in$  all_nodes to 0
43:   for each (src, dst)  $\in$  edges do
44:     outgoing_edges[src]  $\leftarrow$  outgoing_edges[src] + 1
45:   end for
46:   return {node  $\in$  all_nodes | outgoing_edges[node] = 0}
47: end function
48: function ADDDUMMYNODES(layers, edges)
49:   node_to_layer  $\leftarrow$   $\emptyset$ 
50:   for i = 0 to |layers| - 1 do
51:     for each node  $\in$  layers[i] do
52:       node_to_layer[node]  $\leftarrow$  i
53:     end for

```

For time complexity $O(n \cdot e)$ in the worst-case (possibly dominating when removing one node per iteration).

5.2.4 Crossing minimization

Algorithm 15 Minimizing Edge Crossings

```

1: function GETNODEORDER(layer)
2:   order  $\leftarrow \emptyset$ 
3:   for idx = 0 to |layer| - 1 do
4:     order[layer[idx]]  $\leftarrow$  idx
5:   end for
6:   return order
7: end function
8: function GETBARYCENTERPOSITION(node, connected_nodes, node_order)
9:   if connected_nodes =  $\emptyset$  then
10:    return  $\infty$ 
11:   end if
12:   return  $\frac{\sum_{n \in \text{connected\_nodes}} \text{node\_order}[n]}{|\text{connected\_nodes}|}$ 
13: end function
14: function GETMEDIANPOSITION(node, connected_nodes, node_order)
15:   if connected_nodes =  $\emptyset$  then
16:    return  $\infty$ 
17:   end if
18:   positions  $\leftarrow$  sort([node_order[n] for each n  $\in$  connected_nodes])
19:   n  $\leftarrow$  |positions|
20:   if n mod 2 = 1 then
21:     return positions[ $\lfloor n/2 \rfloor$ ]
22:   else
23:     return  $\frac{\text{positions}[(n-1)/2] + \text{positions}[n/2]}{2}$ 
24:   end if
25: end function
26: function MINIMIZECROSSINGS(layers, edges, max_iterations, method)
27:   previous_down_crossings  $\leftarrow \infty$ 
28:   previous_up_crossings  $\leftarrow \infty$ 
29:   successors  $\leftarrow \emptyset$ 
30:   predecessors  $\leftarrow \emptyset$ 
31:   for each layer  $\in$  layers do
32:     for each node  $\in$  layer do
33:       successors[node]  $\leftarrow \emptyset$ 
34:       predecessors[node]  $\leftarrow \emptyset$ 
35:     end for
36:   end for
37:   for each (source, target)  $\in$  edges do
38:     successors[source]  $\leftarrow$  successors[source]  $\cup$  {target}
39:     predecessors[target]  $\leftarrow$  predecessors[target]  $\cup$  {source}
40:   end for
41:   position_func  $\leftarrow$  method = 'barycenter'?GetBarycenterPosition : GetMedianPosition
42:   for iteration = 0 to max_iterations - 1 do
43:     improved  $\leftarrow$  false

```

▷ Sweep down

```

44:   for i = 1 to |layers| - 1 do
45:     fixed_order  $\leftarrow$  GetNodeOrder(layers[i - 1])
46:     current_layer  $\leftarrow$  layers[i]
47:     positions  $\leftarrow \emptyset$ 
48:     for each node  $\in$  current_layer do
49:       pos  $\leftarrow$  position_func(node, predecessors[node], fixed_order)
50:       positions  $\leftarrow$  positions  $\cup$  {(pos, node)}
51:     end for
52:     positions  $\leftarrow$  sort(positions)

```

. Time complexity here is approximately $O(I \cdot L \cdot n^2)$ in the worst-case (especially if counting crossings is quadratic per layer pair).

5.2.5 Node Positioning

Algorithm 16 Node Positioning

```

1: function BUILDPATHSFROMLAYER0(positions, modified_edges, dummy_mapping, max_depth)
2:   layer_0_nodes  $\leftarrow$  [node : (node, (x, y))  $\in$  positions, int(y) = 0]
3:   graph  $\leftarrow$  defaultdict(list)
4:   for each (src, dst)  $\in$  modified_edges do
5:     graph[src].append(dst)
6:   end for
7:   dummy_nodes  $\leftarrow$  set(dummy_mapping.keys())
8:   node_layers  $\leftarrow$  {node : int(y) : (node, (x, y))  $\in$  positions}
9:   all_paths  $\leftarrow$  []
10:  function FINDALLPATHS(source, current_path = None, visited = None)
11:    if current_path = None then
12:      current_path  $\leftarrow$  [source]
13:    end if
14:    if visited = None then
15:      visited  $\leftarrow$  {source}
16:    end if
17:    if max_depth  $\neq$  None and |current_path|  $\geq$  max_depth then
18:      if |current_path| > 1 then
19:        all_paths.append({source : current_path[0], target : current_path[-1], path :
20:          current_path,
21:          dummy_nodes : [n : n  $\in$  current_path, n  $\in$  dummy_nodes], length :
22:          |current_path|})
23:      end if
24:      return
25:    end if
26:    if graph[current_path[-1]] =  $\emptyset$  then
27:      if |current_path| > 1 then
28:        all_paths.append({source : current_path[0], target : current_path[-1], path :
29:          current_path,
30:          dummy_nodes : [n : n  $\in$  current_path, n  $\in$  dummy_nodes], length :
31:          |current_path|})
32:      end if
33:      return
34:    end if
35:    current_node  $\leftarrow$  current_path[-1]
36:    current_layer  $\leftarrow$  node_layers[current_node]
37:    for each neighbor  $\in$  graph[current_node] do
38:      if neighbor  $\in$  visited then
39:        continue
40:      end if
41:      if node_layers.get(neighbor,  $\infty$ )  $\leq$  current_layer then
42:        continue
43:      end if
44:      current_path.append(neighbor)
45:      visited.add(neighbor)
46:      FINDALLPATHS(source, current_path, visited)
47:      current_path.pop()
48:      visited.remove(neighbor)
49:    end for
50:  end function
51:  for each node  $\in$  layer_0_nodes do
52:    FINDALLPATHS(node)
53:  end for
54:  return all_paths

```

Overall Complexity for node postioning: $O(n \log n + I \cdot (P \cdot p + n))$. Since the number of paths P might be large (and is produced by the DFS in `BuildPathsFromLayer0`), it can be the dominating factor in worst-case scenarios.

5.2.6 Edge Reversing and Drawing

Algorithm 17 Draw Smoothed Edges with Dummy Nodes

```

1: procedure BÉZIER CURVE(edge_paths, node_positions, original_edges)
2:   for each (src, dst) and dummies in edge_paths do
3:     if src ∈ node_positions and dst ∈ node_positions then
4:       reverse_direction ← false
5:       if original_edges ≠ null and (dst, src) ∈ original_edges and (src, dst) ∉
         original_edges then
6:         reverse_direction ← true
7:       end if
8:       if reverse_direction then
9:         path_nodes ← [dst] + reverse(dummies) + [src]
10:      else
11:        path_nodes ← [src] + dummies + [dst]
12:      end if
13:      path_points ← GetPositions(path_nodes, node_positions)
14:      if |path_points| < 2 then
15:        continue
16:      end if
17:      if |path_points| ≤ 4 then
18:        points ← ToArray(path_points)
19:        if |points| = 2 then
20:          codes ← [MOVETO, LINETO]
21:        else if |points| = 3 then
22:          codes ← [MOVETO, CURVE3, CURVE3]
23:        else if |points| = 4 then
24:          codes ← [MOVETO, CURVE4, CURVE4, CURVE4]
25:        end if
26:        path ← CreatePath(points, codes)
27:        DrawPath(path)
28:        interpolated ← InterpolatePath(path, 50)
29:      else
30:        points ← ToArray(path_points)
31:        (tck, u) ← SplPrep(points, smoothing = 0.001)
32:        u_new ← LinearSpace(0, 1, 100)
33:        (x_new, y_new) ← SplEv(u_new, tck)
34:        DrawLine(x_new, y_new)
35:        interpolated ← ColumnStack(x_new, y_new)
36:        DrawPiecewiseLinear(points)
37:        interpolated ← points
38:      end if
39:      if |interpolated| ≥ 2 then
40:        end_idx ← |interpolated| − 1
41:        dir_idx ← max(0, end_idx − 5)
42:        end ← interpolated[end_idx]
43:        before_end ← interpolated[dir_idx]
44:        direction ← end − before_end
45:        length ←  $\sqrt{\sum \text{direction}^2}$ 
46:        if length > 0 then
47:          direction ← direction / length
48:          node_radius ← 0.15
49:          arrow_pos ← end − direction × node_radius
50:          arrow_size ← 0.01
51:          DrawArrow(arrow_pos, direction, arrow_size)
52:        end if

```

5.3 Result and Quality Assessment

5.3.1 Results of the Sugiyama Framework

The results represents the final output all the steps involved in sugiyama Framework. The data set used here the Small Directed Network. The first image 'a' shows a circular visulaization of the the Small Directed Network. In the next step of resolving cycles, the 5 edges are chosen to be reversed and they are denoted by the red color. After implementing the Layer Assignment, the number of edge crossing came upto 85. After the cross minimization step, the overall edge crossing came down to 5(barrycenter heuristic). The Node Positioning step minimized the deviation of paths from straight lines using quadratic programming. Finally in the last step, the dummy nodes were removed and the edges(in blue) were drawn replacing them along with reversal of the edges. The edges which were to be reversed were stored initially in step of Resolve cycles.

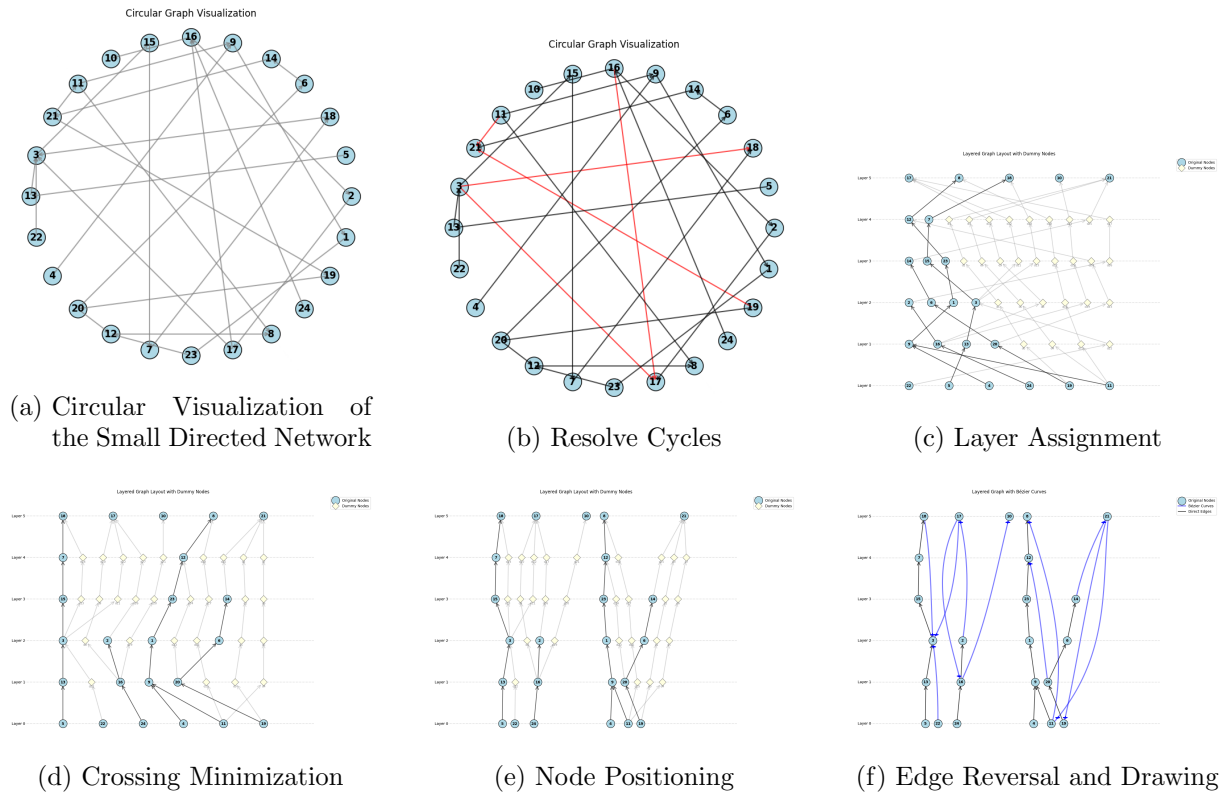


Figure 10: Our implementation of the Sugiyama Framework (Sugiyama, Tagawa, Toda 1981)

5.3.2 Crossing minimization

The algorithm performs iterative sweeps through the graph's layers (both downward and upward) to optimize node ordering and reduce edge crossings. In each iteration, nodes within a layer are reordered based on the positions of their connected nodes in previous layers, using either the barycenter (average position) or median position of their neighbors. The process tracks crossing counts after each sweep and stops when either: 1. no further improvements are possible, 2. the same crossing pattern repeats, or 3. the maximum number of iterations is reached. Considering both downward and upward sweep as one iteration and while using barycenter, the algorithm stops after 3 iteration. The initial number of crossing is 85, after 1st iteration it is 12, 2nd iteration has 7 crossing and the final iteration has 5 crossing. The results were the same when using median.

5.4 Bigger Graphs

When using bigger graphs, we had some issue with node positioning, where it took a very long time to compile, and we were unable to get results for it, but we have the results up until cross minimization as shown in fig 11 .

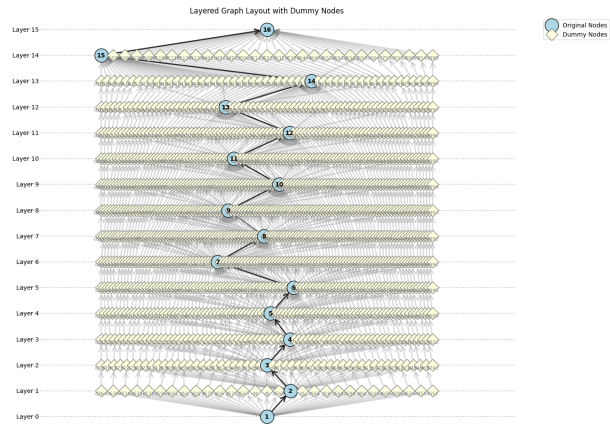


Figure 11: League Network

6 Step 5

6.1 Aim

This step showcases the implementation of Force-Directed Layout on subsets of nodes of a graph (subgraphs), which are drawn within bounding boxes and connected via inter-layer edges. Furthermore, Edge Bundling and Smoothing is applied to these edges, in order to highlight and beautify the connections of different layers of a graph, in this case, its subgraphs.

6.2 Implementation

The following presented function can all be checked in our group's repository.

6.2.1 Loading input graphs and their subgraphs

The `process_dot_file_for_subgraphs()` function extracts and organizes subgraphs from DOT files. Using the `pydot` library, it parses the input file and filters subgraphs according to the specified names. The function systematically extracts all nodes with their attributes from each target subgraph, storing them in a dictionary indexed by the name of the subgraph. To manage edge connections, it creates a comprehensive set of node IDs to efficiently filter edges, retaining only those where both endpoints exist in the selected subgraphs. This approach captures both intra-subgraph edges (within a subgraph) and inter-subgraph edges (between different subgraphs), while preserving all edge attributes. The function returns a structured dictionary containing `'subgraphs'` (mapping each subgraph name to its nodes and attributes) and `'edges'` (representing all connections within and between subgraphs), providing a complete representation ready for visualization.

6.2.2 Applying Force-Directed Layout to Subgraphs within Bounding Boxes

The `draw_colorful_FDG_FR_inertia_subgraphs()` function implements a new version of the Force-Directed Layout (FDL) algorithm for multiple subgraphs. In addition to what was developed for Step 3, this version incorporates the inertia of each node into the modeling of the attractive forces.

The algorithm starts with allocating distinct regions on the canvas for each subgraph based on their node count, with user-adjustable spacing controlled by the `subgraph_spacing` parameter. It then applies the FDL independently to each subgraph, calculating three types of forces: repulsive forces between nodes in the same subgraph, attractive forces exclusively for intra-subgraph edges, and containment forces that keep nodes within their designated regions.

A critical design choice was excluding inter-subgraph edges from attractive force calculations, ensuring subgraphs maintain their assigned positions regardless of interconnections. Furthermore, the convergence of each subgraph is tracked independently, allowing the computation to terminate early for stabilized components.

The visualization renders rectangular bounding boxes for each subgraph region, convex hull polygons around node clusters, and nodes colored according to subgraph membership. The process continues until all subgraphs converge or the maximum iteration count is reached, producing a clear visualization of complex multi-subgraph structures while preserving inter-subgraph relationship visibility.

6.2.3 Applying Force Directed Edge Bundling

This part uses the concept of "Force Directed Edge Bundling for graph visualization" [2] by Danny Holten and Jarke J. van Wijk. The Force-Directed Edge Bundling (FDEB) method organizes similar edges in graph visualizations through bundling techniques to reduce visual

complexity. The algorithm applies a spring-mechanism to edges which enables them to bend through attraction. Every edge gets split up into sections referred to as control points. The algorithm selects edges which have equivalent directional orientations and lengths and similar placement positions. Within the algorithm two main forces operate: spring forces to preserve edge forms combined with attraction forces that minimally distant compatible edges. The iteration process leads to the generation of smooth bundled curves. Implementation of functionality in the code allows users to bundle specific edges between clusters in order to display group relationships clearly without cluttering the visualization. Graph understanding becomes simpler through the application of this technique.

Algorithm 18 Force-Directed Edge Bundling (FDEB)

```

1: procedure FDEB( $G, pos, P = 7, S = 0.03, I = 50, \tau = 0.06$ )
2:   Compatibility Functions:
3:    $C_{angle}(e_1, e_2) \leftarrow |\cos(\angle(e_1, e_2))|$ 
4:    $C_{scale}(e_1, e_2) \leftarrow 2/(l_1/l_2 + l_2/l_1)$ 
5:    $C_{pos}(e_1, e_2) \leftarrow l_1/(l_1 + \|mid_1 - mid_2\|)$ 
6:    $C_{vis}(e_1, e_2) \leftarrow \max(0, 1 - 2d_1/l_2)$ 
7:    $C(e_1, e_2) \leftarrow C_{angle} \times C_{scale} \times C_{pos} \times C_{vis}$ 
8:   Forces:
9:    $F_{spring}(p, p', p'', l_0) \leftarrow \sum_{n \in \{p', p''\}} \frac{n-p}{\|n-p\|} \times (\|n-p\| - l_0)$ 
10:   $F_{elec}(p, q) \leftarrow k \cdot \frac{q-p}{\|q-p\|^2}$ 
11:  Initialize:
12:   $CP_e \leftarrow$  Distribute  $(P + 2)$  points along each edge  $e$ 
13:   $CE \leftarrow$  Find compatible edges where  $C(e_1, e_2) > \tau$ 
14:  Main Loop:
15:  for  $iteration \leftarrow 1$  to  $I$  do
16:    for each edge  $e$  do
17:       $l_0 \leftarrow \|e\|/(P + 1)$  ▷ Rest length
18:      for  $i \leftarrow 1$  to  $P$  do ▷ For each control point except endpoints
19:         $F \leftarrow F_{spring}(CP_e[i], CP_e[i - 1], CP_e[i + 1], l_0)$ 
20:        for each compatible edge  $e'$  of  $e$  do
21:           $F \leftarrow F + F_{elec}(CP_e[i], CP_{e'}[i])$ 
22:        end for
23:         $CP_e[i] \leftarrow CP_e[i] + S \cdot F$  ▷ Update position
24:      end for
25:    end for
26:  end for
27:  return  $CP$  ▷ Return control points for all edges
28: end procedure
29: procedure APPLYFDEBBETWEENCLUSTERS( $G, pos, c_1, c_2$ )
30:   $E_{inter} \leftarrow \{(u, v) \in G.edges() : (u \in c_1 \wedge v \in c_2) \vee (u \in c_2 \wedge v \in c_1)\}$ 
31:   $G_{sub} \leftarrow G.edge\_subgraph(E_{inter})$ 
32:  return  $FDEB(G_{sub}, pos)$ 
33: end procedure

```

6.2.4 Applying Smoothing

The smoothing technique for edge bundling in graph visualizations was used to reduce jaggedness in bundled edges by applying Gaussian convolution to smooth the control points along the edges. The `smooth_edge` function smooths the x and y coordinates of the edge points using a Gaussian

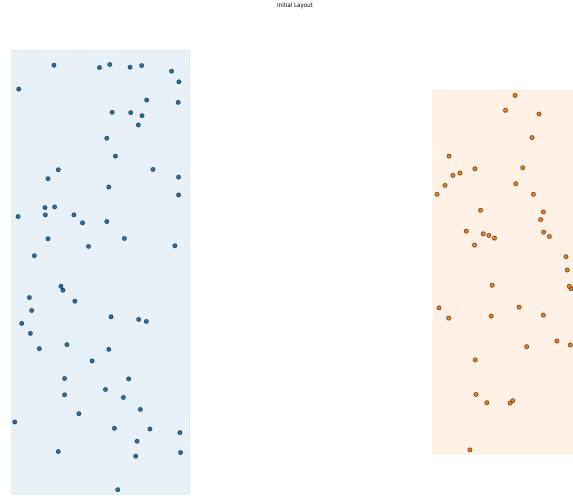


Figure 12: Initial Positions

kernel, while `sample_edge` ensures that the points are equally spaced after smoothing. The `smooth_bundled_edges` function applies the Gaussian smoothing to all control points of bundled edges, keeping the endpoints fixed. The `resample_edge_points` function resamples the points to maintain equal spacing along the edges. The `apply_fdeb_with_smoothing` function integrates Force-Directed Edge Bundling (FDEB) with smoothing, first applying FDEB to inter-cluster edges and then smoothing and resampling the results. The Key parameters include *kernel_width* (width of the Gaussian kernel), *sigma* (standard deviation of the kernel), and *smooth_iterations* (number of smoothing passes). Once the initial smoothing was done, we repeated the process 10 times with the function `edge_smoothing_with_iterations` until we had an optimal output.

6.3 Results and Quality Assesment

In our case, we run the algorithm on the Argumentation Network with "Youngest Devonian Strata" and "Gap in the Sequence of Devonshi" as subgraphs.

Initially as shown in the first output, the nodes are positioned randomly and the subgraph is given its own color blue and orange.

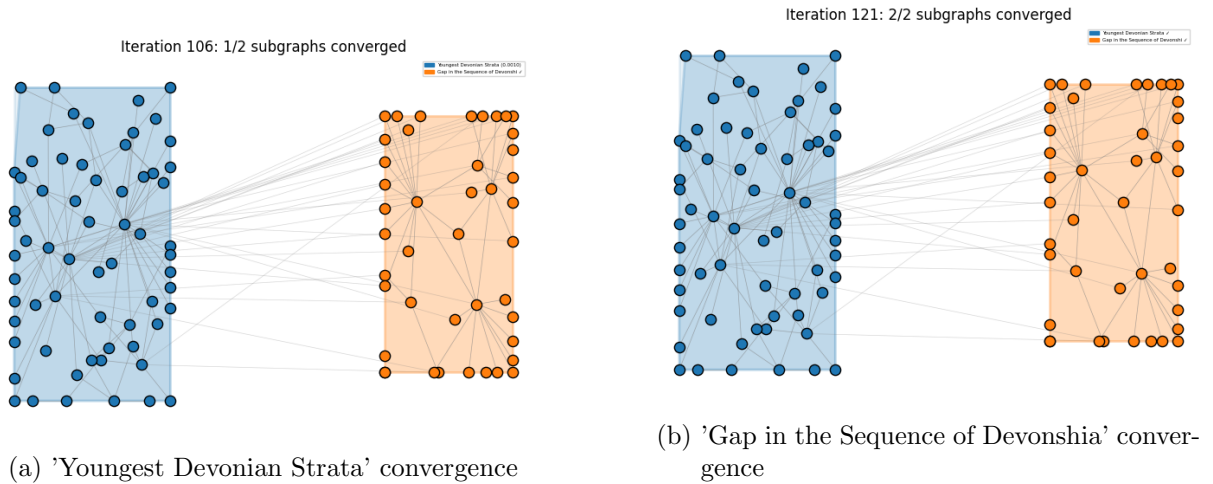


Figure 13: Convergence analysis of Devonian strata features

The FDL algorithm is applied next, where the Subgraph 'Youngest Devonian Strata' con-

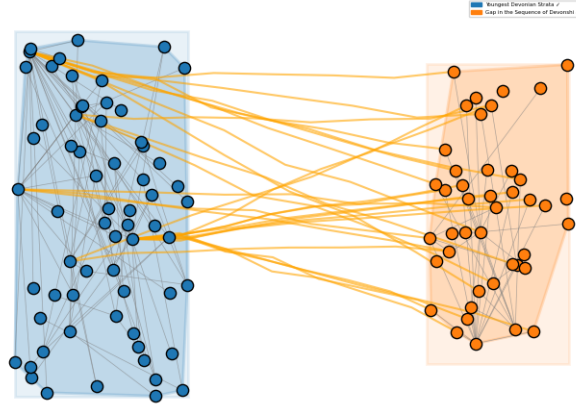


Figure 14: Edge Bundling

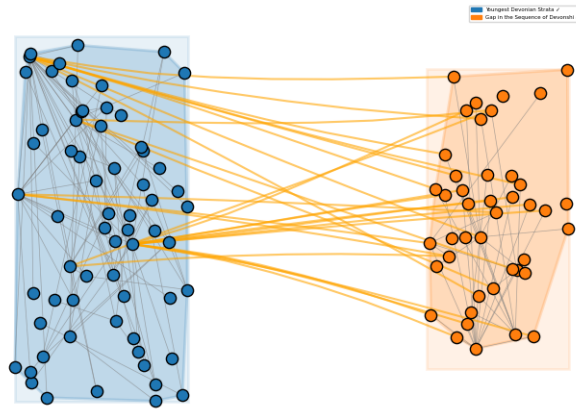


Figure 15: Initial Smoothing

verged at iteration 106 with max movement 0.000498 and the Subgraph 'Gap in the Sequence of Devonshi' converged at iteration 121 with max movement 0.000486.

The visualization in Figure 14 demonstrates the results of applying Force-Directed Edge Bundling (FDEB) with parameters set to $P=6$ (control points per edge), $S=0.03$ (step size for force calculations), $I=50$ (iterations), and a compatibility threshold of 0.06. The output shows successful edge bundling, where similar edges are grouped into smooth, curved paths that clearly reveal connection patterns between nodes. While most edges form coherent bundles, some portions exhibit jaggedness. These are resolved by implementing Gaussian smoothing. The visualization effectively reduces visual clutter compared to traditional straight-line edge drawings, making identifying connections between the nodes easier.

Once smoothing was applied, as seen in Figure 15, there is an improvement in the edge bundling visualization by eliminating the jagged artifacts present in the initial output. After applying Gaussian smoothing to the edge paths, the bundled connections now appear as connections that maintain their directional flow while appearing more natural and visually pleasing. The smoothing was applied with 2 iterations, a kernel width of 5, and $\sigma = 1$, effectively reducing jagged edges. The technique has been particularly effective in areas where multiple edges converge, creating smooth bundled connections rather than the previous disjointed segments.

7 Step 6

7.1 Aim

In this step we compute distance matrices using graph-theoretic distance. Then these matrices are used as input by MDS, t-SNE, and ISOMAP to reduce dimensionality and project the graph's drawings on a 2D/3D canvas: first the nodes are drawn as blue circles with their ids nearby and then gray edges are drawn between connected nodes.

7.2 Implementation

The following presented functions can all be checked in our group's repository.

7.2.1 Floyd-Warshall Algorithm and Distance Matrix Visualization

The Floyd-Warshall algorithm is used to compute the shortest paths between all pairs of nodes in a weighted graph, represented by `nodes_dict` and `edges_dict`. The algorithm begins by initializing a distance matrix with ∞ for all non-adjacent nodes and zero for diagonal elements (distance from a node to itself). Edge weights are then populated into the matrix, with a default weight of 1 if no weight is specified.

The implementation includes a `proximity` parameter to handle graphs where weights represent connectivity strength rather than distance. When `proximity=True`, the weights are transformed using the formula `new_weight = max_weight - original_weight` to convert from proximity to distance metrics. Here, `original_weight` represents the proximity value (higher values indicate stronger connections or closer nodes), `max_weight` is the maximum proximity value found in the graph, and `new_weight` becomes the distance measure (higher values indicate greater separation between nodes).

The algorithm consists of three nested loops: the outer loop iterates over each node as a potential intermediate, while the inner loops update the shortest path distances between all pairs of nodes by checking if a path through the intermediate node yields a shorter distance. The final distance matrix is returned, along with the list of node IDs.

To facilitate interpretation of these graph-theoretic distances, we visualize the distance matrix using a color-coded heatmap through the `plot_coloured_distance_matrix()` function. This visualization maps the numerical distance values to a color gradient, with darker colors representing longer distances and lighter colors indicating shorter paths between pairs of nodes.

7.2.2 MDS

The Multidimensional Scaling (MDS) is a dimensionality reduction technique that aims to preserve the pairwise distances between data points when projecting high-dimensional data into a lower-dimensional space. In this step, MDS was applied to a distance matrix derived from the dataset, reducing it to two dimensions (`n_components = 2`). The MDS algorithm works by minimizing the stress function, which quantifies the difference between the original high-dimensional distances and the distances in the reduced space.

7.2.3 t-SNE

This t-SNE (t-distributed Stochastic Neighbor Embedding) algorithm begins by processing the input distance matrix. To ensure optimal visualization across different graph sizes, the `perplexity` parameter is dynamically adjusted. For graphs with fewer than 100 nodes, perplexity ranges from 5 to 12. For larger graphs, perplexity is set between 5% and 10% of the total node count. [3] details how using perplexity as a percentage of the dataset (e.g., 5%–10%) can be effective, especially when considering density and noise in

the data. The distance metric is set to ‘precomputed’ since we already have the graph distance matrix. The `angle` (Barnes-Hut Parameter) is set to 0.2 for increased accuracy, as explained in [4]. We also tried other `angle` values between 0.2 and 0.8, but did not notice any significant changes in the computation time and overall quality of the projection. `Random_state` is set to 42 for reproducibility, the `learning_rate` is set to ‘auto’ to automatically adjust based on the sample size, and the `n_components` is set to 2 for a 2D visualization.

7.2.4 ISOMAP

The ISOMAP (Isometric Mapping) assumes that the data lies on a lower-dimensional manifold (curved surface) embedded in the high-dimensional space. It calculates distances along this manifold rather than direct Euclidean distances in $\mathbb{R}^d$, making it more flexible for capturing nonlinear structures. In our implementation, we applied ISOMAP to the precomputed distance matrix with both 2 and 3 components to obtain both 2D and 3D representations of the data manifold. The `metric` parameter was set to “precomputed” since the graph distance matrix is given as input. To understand how the manifold structure emerges, we varied the `n_neighbors` parameter from 1 to the total number of nodes minus one. This allowed us to observe the convergence behavior of the manifold reconstruction as more neighboring relationships were considered.

7.3 Results and Quality Assessment

7.3.1 Distance Matrix

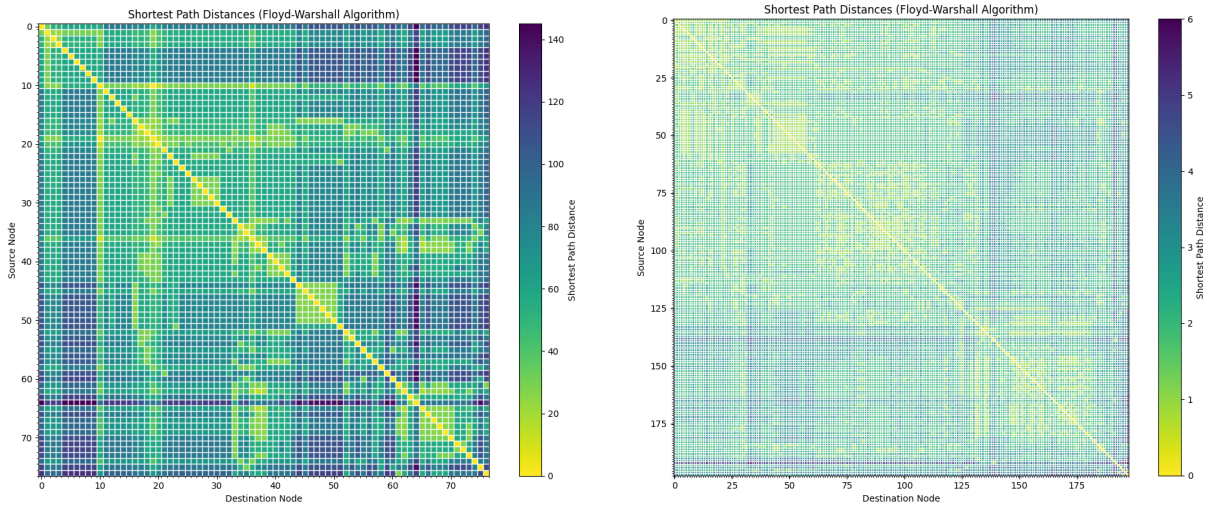


Figure 16: Distance matrix visualizations for LesMiserables (left) and JazzNetwork (right)

In Figure 21 we can see how the color-coded visualization enables quick identification of potential clusters and community structures before the applications of dimensionality reduction techniques. Comparing different datasets revealed distinct structural patterns: the Jazz Network exhibited predominantly lighter colors, indicating that most nodes maintain close proximity to each other in a tightly connected community, while the Les Misérables network had more pronounced dark regions, suggesting separation between groups of nodes.

7.3.2 MDS

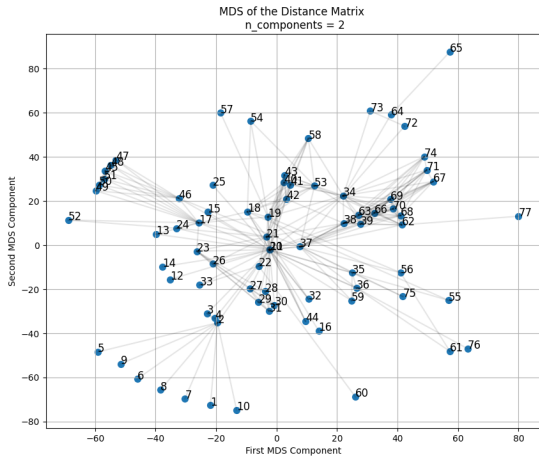


Figure 17: Les Misérables

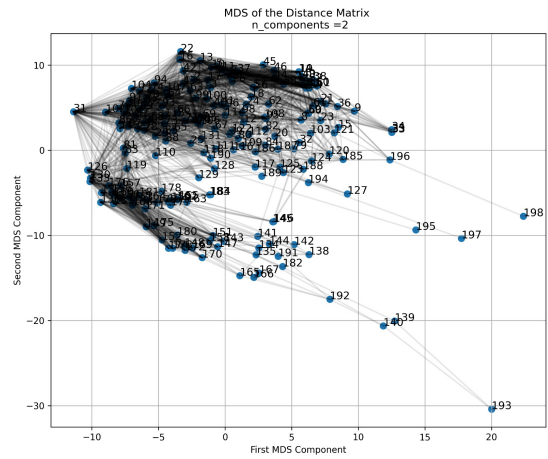


Figure 18: Jazz Network

As seen in Figure 17 and 18, the MDS visualization projects the data while preserving pairwise distances between nodes, revealing groupings where closely positioned nodes present higher similarity. The plot shows distinct clusters in certain regions, where groups of nodes with shared structural or relational properties are present. At the same time, other areas exhibit sparser distributions, with unique or outlier nodes that differ from the rest. Similarly in the Jazz network, the densely clustered and tightly grouped nodes suggesting strong similarities or relationships between the nodes. There seems to be only few nodes isolated to the end of the plot. The high cluttering of the edges in the projection show a strong connection between the nodes whereas the isolated nodes do not have a lot of connections.

7.3.3 t-SNE

The t-SNE algorithm attempts to maintain the similarities in 2D space, where nodes with small shortest path distances are close in visualization. The nodes in the same community are likely clustered together, distant nodes in the graph are separated in visualization.

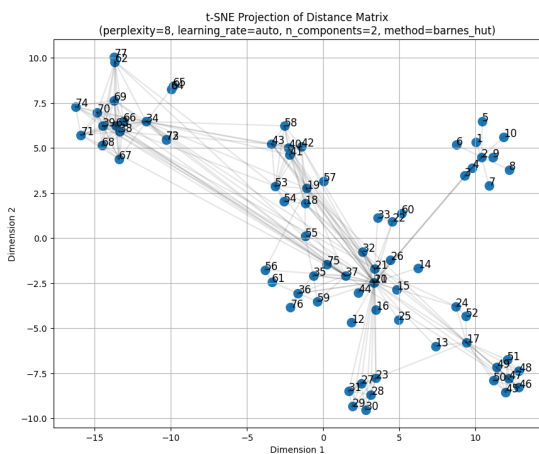


Figure 19: Les Misérables

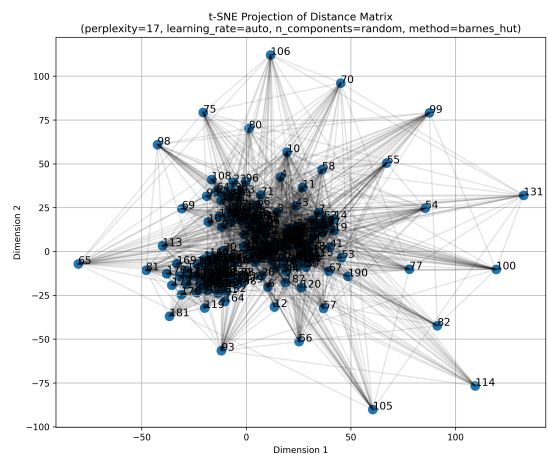


Figure 20: Jazz Network

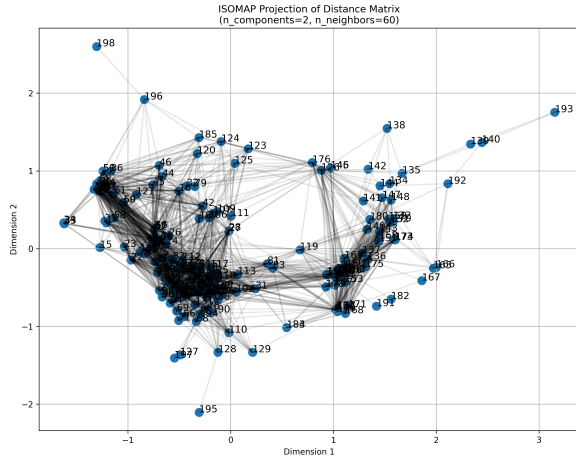
In Jazz Network, Figure 20, different parameters gave slightly different node positions, but the above output gave distinct separation between dissimilar nodes, forming two small groups near the center. In Les Misérables, Figure 19, the clustering of the nodes were very similar for

different perplexity values we chose, but the value 8 gave a good distinction between different clusters.

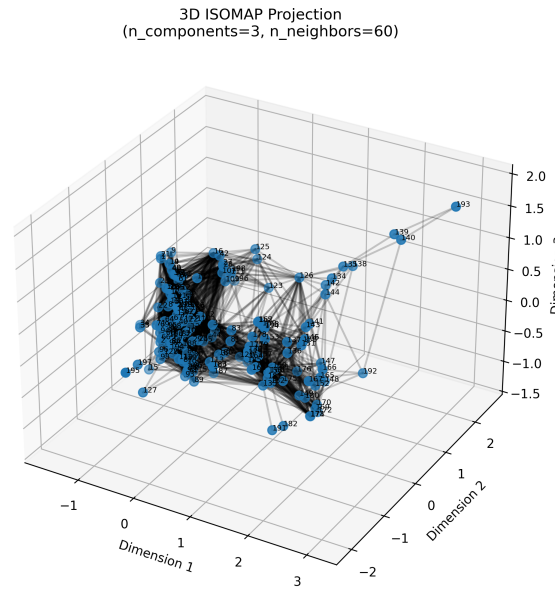
7.3.4 ISOMAP

In the ISOMAP projections, Figure 21, some interesting convergence behaviors emerged by varying `n_neighbors`. For the Les Misérables graph, we observed that the projection stabilized after approximately 12 neighbors in both 2D and 3D representations, with minimal changes beyond this threshold. In contrast, the Jazz Network required significantly more neighbors to achieve stability, beginning to converge around 30 neighbors and reaching a practically unchanged configuration after surpassing 60 neighbors. This difference in convergence behavior likely derives from the Jazz Network’s more densely connected structure with many closely positioned nodes, necessitating a larger neighborhood to accurately capture the underlying graph topology.

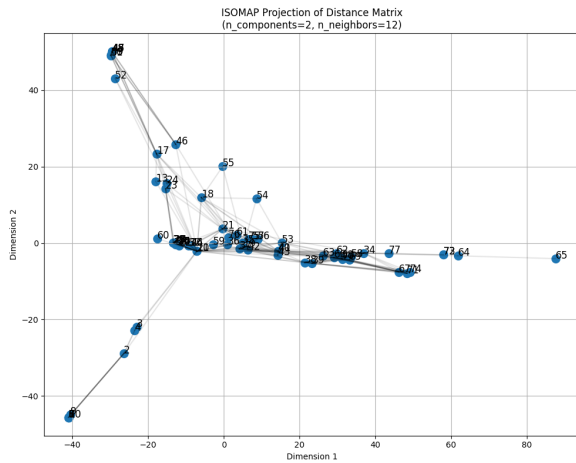
However, while a pronounced manifold structure was not immediately evident in most projections, the Jazz Network revealed subtle indications of a curved surface when visualized in 3D ISOMAP. This curvature appeared to be further unfolded when projected to 2D, suggesting the presence of a weak manifold structure that ISOMAP was partially able to capture. This contrasts with Les Misérables, which exhibited little evidence of underlying manifold properties. Hence, the ISOMAP may not be the most appropriate dimensionality reduction algorithm for the latter graph.



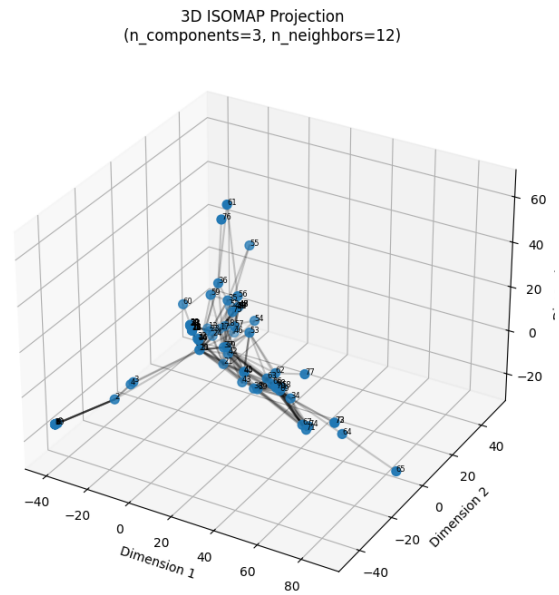
(a) ISOMAP 2D projection of JazzNetwork with 60 neighbors considered



(b) ISOMAP 3D projection of JazzNetwork with 60 neighbors considered



(c) ISOMAP 2D projection of LesMiserable with 12 neighbors considered



(d) ISOMAP 3D projection of LesMiserable with 12 neighbors considered

Figure 21: ISOMAP projections of JazzNetwork (above) and LesMiserable (below) after convergence

8 Step 7

8.1 Aim

This step computes the quality metrics such as crossing number, crossing resolution, stress of layout for force-directed and layered layout. For projections continuity, trustworthiness, stress, Shepard quality metrics are computed.

8.2 Quality metrics for Graph Visualizations

These Quality Metrics were computed for the Force Directed Layout Graphs and the Layered Layout Graphs.

8.2.1 Number of crossing

The 'countedges crossings final' function implements edge crossing detection in graph layouts by analyzing pairs of edges for intersections. It takes node positions and edges as input, using geometric calculations to determine if two line segments cross. The algorithm employs a helper function 'linesents intersect' that calculates orientations of point triplets to detect intersections, handling both standard crossings and special collinear cases. The function excludes cases where edges share endpoints and maintains both a count of crossings and a list of crossing edge pairs.

“

8.2.2 Crossing Resolution

The 'calculate crossing angles' function measures the acute angles between intersecting edges in the graph layout. For each pair of crossing edges identified by the previous crossing detection algorithm, it computes the angle between them using vector mathematics. The function first calculates vectors from the edge endpoints using their coordinates, then determines the angle using the dot product formula and arc cosine function. To ensure accuracy, it handles special cases like zero-length vectors and normalizes the results to return acute angles (90 degrees).

Algorithm 19 Number of Crossings

```
1: procedure COUNTEDGECROSSINGS(positions, edges)
2:   crossings  $\leftarrow$  0
3:   crossing_edges  $\leftarrow$  []
4:   for  $i \leftarrow 0$  to  $|edges| - 1$  do
5:     src1, dst1  $\leftarrow$  edges[ $i$ ]
6:     if src1  $\notin$  positions OR dst1  $\notin$  positions then
7:       continue
8:     end if
9:     p1  $\leftarrow$  positions[src1]
10:    p2  $\leftarrow$  positions[dst1]
11:    for  $j \leftarrow i + 1$  to  $|edges| - 1$  do
12:      src2, dst2  $\leftarrow$  edges[ $j$ ]
13:      if src2  $\notin$  positions OR dst2  $\notin$  positions then
14:        continue
15:      end if
16:      p3  $\leftarrow$  positions[src2]
17:      p4  $\leftarrow$  positions[dst2]
18:      if src1 = src2 OR src1 = dst2 OR dst1 = src2 OR dst1 = dst2 then
19:        continue  $\triangleright$  Edges sharing endpoints don't count as crossings
20:      end if
21:      if LINESEGMENTSINTERSECT(p1, p2, p3, p4) then
22:        crossings  $\leftarrow$  crossings + 1
23:        crossing_edges.APPEND(((src1, dst1), (src2, dst2)))
24:      end if
25:    end for
26:  end for
27:  return crossings, crossing_edges
28: end procedure
29: procedure LINESEGMENTSINTERSECT(p1, p2, p3, p4)
30:   o1  $\leftarrow$  ORIENTATION(p1, p2, p3)
31:   o2  $\leftarrow$  ORIENTATION(p1, p2, p4)
32:   o3  $\leftarrow$  ORIENTATION(p3, p4, p1)
33:   o4  $\leftarrow$  ORIENTATION(p3, p4, p2)
34:   if o1  $\neq$  o2 AND o3  $\neq$  o4 then
35:     return true  $\triangleright$  General case: different orientations
36:   end if
37:   if o1 = 0 AND ONSEGMENT(p1, p3, p2) then
38:     return true
39:   end if
40:   if o2 = 0 AND ONSEGMENT(p1, p4, p2) then
41:     return true
42:   end if
43:   if o3 = 0 AND ONSEGMENT(p3, p1, p4) then
44:     return true
45:   end if
46:   if o4 = 0 AND ONSEGMENT(p3, p2, p4) then
47:     return true
48:   end if
49:   return false  $\triangleright$  No intersection
50: end procedure
51: procedure ORIENTATION(p, q, r)
52:   val  $\leftarrow$  ( $q_y - p_y$ )  $\times$  ( $r_x - q_x$ ) - ( $q_x - p_x$ )  $\times$  ( $r_y - q_y$ )
53:   if val = 0 then
54:     return 0 47  $\triangleright$  Collinear
55:   else if val > 0 then
56:     return 1  $\triangleright$  Clockwise
```

Algorithm 20 Calculate Crossing Angles

```
1: function CALCULATECROSSINGANGLES(positions, crossing_edges)
2:   results  $\leftarrow \emptyset$ 
3:   for each (edge1, edge2)  $\in$  crossing_edges do
4:     Get coordinates of edge1 endpoints ( $p_1, p_2$ )
5:     Get coordinates of edge2 endpoints ( $p_3, p_4$ )
6:     Calculate  $\vec{v}_1 = p_2 - p_1$ 
7:     Calculate  $\vec{v}_2 = p_4 - p_3$ 
8:     Calculate dot_product  $= \vec{v}_1 \cdot \vec{v}_2$ 
9:     Calculate magnitudes of both vectors  $|\vec{v}_1|, |\vec{v}_2|$ 
10:    if either  $|\vec{v}_1| = 0$  or  $|\vec{v}_2| = 0$  then
11:      angle  $\leftarrow 0$ 
12:    else
13:      cos_angle  $\leftarrow \frac{\text{dot\_product}}{|\vec{v}_1| \cdot |\vec{v}_2|}$ 
14:      angle  $\leftarrow \arccos(\text{cos\_angle})$ 
15:      Convert angle to degrees
16:      angle  $\leftarrow \min(\text{angle}, 180 - \text{angle})$ 
17:    end if
18:    Add (edge1, edge2, angle) to results
19:  end for
20:  return results
21: end function
```

8.2.3 Stresslayout

The 'calculate layout stress' first builds an adjacency list representation of the graph and computes shortest paths between all node pairs using Breadth-First Search (BFS). The stress calculation then compares the geometric distances between node positions in the layout against their corresponding shortest path distances. For each node pair, it computes a weighted stress value, where the weight is inversely proportional to the square of the target distance. The final stress value is normalized by the number of node pairs, where lower values indicate better preservation of graph structure in the visualization.

Algorithm 21 Calculate Layout Stress

```
1: function CALCULATELAYOUTSTRESS(positions, edges)
2:   Create adjacency list from edges
3:   for each start_node  $\in$  positions do
4:     Initialize distances using BFS
5:     queue  $\leftarrow [(start\_node, 0)]$ 
6:     while queue not empty do
7:       Get current node and distance
8:       for each neighbor do
9:         if not visited then
10:           Record distance
11:           Add to queue
12:         end if
13:       end for
14:     end while
15:   end for
16:   stress  $\leftarrow 0$ 
17:   pairs_count  $\leftarrow 0$ 
18:   for each node pair (node1, node2) do
19:     if nodes are connected then
20:       Calculate geometric_distance
21:       Get target_distance from BFS results
22:       weight  $\leftarrow \frac{1}{(target\_distance)^2}$ 
23:       stress  $\leftarrow stress + weight \cdot (geometric\_distance - target\_distance)^2$ 
24:       Increment pairs_count
25:     end if
26:   end for
27:   return  $\frac{stress}{pairs\_count}$ 
28: end function
```

8.3 Quality metrics for the Projections for graphs

8.3.1 Trustworthiness and Continuity

The `trustworthiness_and_continuity` function evaluates how well neighborhood relationships are preserved during dimensionality reduction through two complementary metrics. This function takes the original high-dimensional data matrix, its low-dimensional projection, and a neighborhood size parameter `n_neighbors`, and returns scores between 0 and 1 for both metrics.

The implementation begins by calculating pairwise Euclidean distances between all points in both high and low-dimensional spaces. It then identifies the `k`-nearest neighbors for each point in both spaces, which are essential for detecting neighborhood changes. For effective comparison,

the algorithm also calculates rank matrices that represent the position of each point in the distance ordering from every other point.

For trustworthiness calculation, the algorithm identifies "false neighbors" - points that appear close in the low-dimensional projection but weren't neighbors in the original space. For each false neighbor, it computes a penalty based on how far that point was in the high-dimensional ranking. These penalties are summed and normalized to produce the trustworthiness score, with higher values indicating fewer deceptive neighbor relationships.

For continuity, the algorithm identifies "missing neighbors" - points that were neighbors in the high-dimensional space but aren't neighbors in the low-dimensional projection. It then calculates penalties based on how far apart these points have been pushed in the low-dimensional ranking. After normalization, a high continuity score indicates good preservation of original proximity relationships.

Both metrics use the same normalization factor derived from the dataset size and neighborhood parameter, ensuring scores remain in the 0-1 range, where 1 represents perfect neighborhood preservation.

Algorithm 22 Trustworthiness and Continuity

```
1: function TRUSTWORTHINESSANDCONTINUITY( $X_{high}, X_{low}, k$ )
2:    $n \leftarrow$  number of samples
3:   // Compute pairwise distances in high-dimensional space
4:    $D_{high} \leftarrow$  zeros( $n, n$ )
5:   for  $i = 0$  to  $n - 1$  do
6:     for  $j = i + 1$  to  $n - 1$  do
7:        $dist \leftarrow$  EuclideanDistance( $X_{high}[i], X_{high}[j]$ )
8:        $D_{high}[i, j] \leftarrow dist$ 
9:        $D_{high}[j, i] \leftarrow dist$ 
10:    end for
11:  end for
12:  // Compute pairwise distances in low-dimensional space
13:   $D_{low} \leftarrow$  zeros( $n, n$ )
14:  for  $i = 0$  to  $n - 1$  do
15:    for  $j = i + 1$  to  $n - 1$  do
16:       $dist \leftarrow$  EuclideanDistance( $X_{low}[i], X_{low}[j]$ )
17:       $D_{low}[i, j] \leftarrow dist$ 
18:       $D_{low}[j, i] \leftarrow dist$ 
19:    end for
20:  end for
21:  // Find k-nearest neighbors in both spaces
22:   $N_{high} \leftarrow$  zeros( $n, k$ )
23:   $N_{low} \leftarrow$  zeros( $n, k$ )
24:  for  $i = 0$  to  $n - 1$  do
25:     $N_{high}[i] \leftarrow$  indices of k smallest values in  $D_{high}[i]$  excluding  $i$ 
26:     $N_{low}[i] \leftarrow$  indices of k smallest values in  $D_{low}[i]$  excluding  $i$ 
27:  end for
28:  // Calculate ranks in both spaces
29:   $R_{high} \leftarrow$  zeros( $n, n$ )
30:   $R_{low} \leftarrow$  zeros( $n, n$ )
31:  for  $i = 0$  to  $n - 1$  do
32:     $R_{high}[i] \leftarrow$  rank order of  $D_{high}[i]$ 
33:     $R_{low}[i] \leftarrow$  rank order of  $D_{low}[i]$ 
34:  end for
35:  // Calculate trustworthiness
36:   $T_{sum} \leftarrow 0$ 
37:  for  $i = 0$  to  $n - 1$  do
38:     $U_k(i) \leftarrow \{j | j \in N_{low}[i] \text{ and } j \notin N_{high}[i]\}$   $\triangleright$  False neighbors
39:    for  $j \in U_k(i)$  do
40:       $T_{sum} \leftarrow T_{sum} + (R_{high}[i, j] - k)$ 
41:    end for
42:  end for
43:  // Calculate continuity
44:   $C_{sum} \leftarrow 0$ 
45:  for  $i = 0$  to  $n - 1$  do
46:     $V_k(i) \leftarrow \{j | j \in N_{high}[i] \text{ and } j \notin N_{low}[i]\}$   $\triangleright$  Missing neighbors
47:    for  $j \in V_k(i)$  do
48:       $C_{sum} \leftarrow C_{sum} + (R_{low}[i, j] - k)$ 
49:    end for
50:  end for
51:  // Normalize scores
52:   $G \leftarrow 2 / (n \cdot k \cdot (2n - 3k - 1))$ 
53:   $trustworthiness \leftarrow 1 - G \cdot T_{sum}$ 
54:   $continuity \leftarrow 1 - G \cdot C_{sum}$ 
55:  return  $trustworthiness, continuity$ 
56: end function
```

8.3.2 Normalized Stress

Normalized Stress provides a global measure of how well pairwise distances between points are preserved after dimensionality reduction. This metric, derived from Kruskal's stress formula, quantifies the discrepancy between distances in the original high-dimensional space and those in the reduced low-dimensional space. The `normalized_stress` function takes as input the original high-dimensional data matrix and its lower-dimensional projection, then compares corresponding distances. For each pair of points, it calculates their Euclidean distance in both spaces and accumulates the squared differences. These squared differences are summed and then normalized by dividing by the sum of squared original distances. A stress value of 0 indicates perfect distance preservation, while increasing values reflect greater distortion of the original distance relationships.

Algorithm 23 Stress Calculation

```
1: function CALCULATESTRESS( $X_{high}, X_{low}$ )
2:    $n \leftarrow$  number of samples
3:   // Compute all pairwise distances in both spaces
4:    $distances_{high} \leftarrow []$ 
5:    $distances_{low} \leftarrow []$ 
6:   for  $i = 0$  to  $n - 1$  do
7:     for  $j = i + 1$  to  $n - 1$  do
8:        $d_{high} \leftarrow$  EuclideanDistance( $X_{high}[i], X_{high}[j]$ )
9:        $d_{low} \leftarrow$  EuclideanDistance( $X_{low}[i], X_{low}[j]$ )
10:       $distances_{high}.append(d_{high})$ 
11:       $distances_{low}.append(d_{low})$ 
12:    end for
13:  end for
14:  // Calculate Kruskal's stress formula
15:   $numerator \leftarrow \sum_i (distances_{high}[i] - distances_{low}[i])^2$ 
16:   $denominator \leftarrow \sum_i distances_{high}[i]^2$ 
17:   $stress \leftarrow \sqrt{numerator/denominator}$ 
18:  return  $stress$ 
19: end function
```

8.3.3 Shepard Diagram

Shepard Diagram offers a visual assessment of distance preservation in dimensionality reduction, complementing numeric metrics like Normalized Stress by showing visual patterns in how original and projected distances relate to each other. This visualization tool takes as input both the original high-dimensional data matrix and its low-dimensional projection, then produces a scatter plot that reveals the relationship between distances in the two spaces. The `plot_shepard_diagram` function systematically calculates Euclidean distances between all pairs of points in both spaces, then plots each pair as a point where the x-coordinate represents the original distance and the y-coordinate represents the corresponding distance in the reduced space. The visualization includes a diagonal reference line representing perfect distance preservation (where $y=x$), as well as a regression line that indicates the overall trend of the plotted points. The correlation coefficient (r-value) provides a numerical measure of how linearly the distances are related. In an ideal projection, points would cluster tightly around the bisector, indicating that distances are accurately maintained.

Algorithm 24 Shepard Diagram

```
1: function CREATESHEPARDDIAGRAM( $X_{high}, X_{low}$ )
2:    $n \leftarrow$  number of samples
3:    $original\_distances \leftarrow []$ 
4:    $projected\_distances \leftarrow []$ 
5:   for  $i = 0$  to  $n - 1$  do
6:     for  $j = i + 1$  to  $n - 1$  do
7:        $d_{high} \leftarrow$  EuclideanDistance( $X_{high}[i], X_{high}[j]$ )
8:        $d_{low} \leftarrow$  EuclideanDistance( $X_{low}[i], X_{low}[j]$ )
9:        $original\_distances.append(d_{high})$ 
10:       $projected\_distances.append(d_{low})$ 
11:    end for
12:  end for
13:  // Create scatter plot
14:   $fig, ax \leftarrow$  CreatePlot()
15:   $ax.ScatterPlot(original\_distances, projected\_distances)$ 
16:  // Add diagonal reference line (y=x)
17:   $max\_dist \leftarrow \max(\max(original\_distances), \max(projected\_distances))$ 
18:   $ax.PlotLine([0, max\_dist], [0, max\_dist])$ 
19:  // Calculate and plot regression line
20:   $slope, intercept, r\_value \leftarrow$  LinearRegression( $original\_distances, projected\_distances$ )
21:   $fitted\_line \leftarrow slope \cdot original\_distances + intercept$ 
22:   $ax.PlotLine(original\_distances, fitted\_line)$ 
23:   $ax.AddText("Correlation:  $r\_value$ ")$ 
24:   $ax.SetLabels("Original Distances", "Projected Distances")$ 
25:   $ax.SetTitle("Shepard Diagram")$ 
26:  return  $fig, ax$ 
27: end function
```

8.4 Results and Discussion

8.5 Force Directed Layouts

For the computation of the quality metrics for the force-directed layouts, we used the Parameters: 300 iterations, 0.001 convergence threshold, C values (0.1, 0.2, 0.75, 1.5). the Lower C values produced more compact layouts, while higher C values pushed nodes to edges and increased convergence time. From the parameters and the 3 data sets used, we got the following outputs.

Table 1: Network Performance Metrics for Different C Values

Network & Parameter	Crossing Number	Crossing Resolution	Stress
Jazz Network			
C = 0.1	403,576	0.001428°	20,607.998
C = 0.2	331,670	0.002174°	18,788.322
C = 0.75	391,236	0.013350°	13,344.029
C = 1.5	416,653	0.003290°	8,014.458
Les Miserables Network			
C = 0.1	2,319	0.027077°	2,694.262
C = 0.2	2,255	0.895713°	2,140.130
C = 0.75	2,097	0.508919°	1,508.908
C = 1.5	2,359	0.510027°	1,183.742
League Network			
C = 0.1	3,456	0.001138°	177.910
C = 0.2	3,968	49.901303°	170.411
C = 0.75	4,082	49.690016°	127.210
C = 1.5	4,324	65.062745°	78.870

The analysis of our three networks reveals distinct patterns in how the parameter C affects layout quality metrics. In the Jazz Network, C=0.2 yielded the lowest crossing number while C=1.5 produced the highest, yet stress consistently decreased as C increased, indicating a clear trade-off between crossing minimization and stress reduction. The Les Miserables Network showed optimal crossing minimization at the mid-range value of C=0.75, while maintaining the same inverse relationship between C and stress. Interestingly, the League Network exhibited contrary behavior with crossing numbers increasing alongside higher C values.

8.5.1 Layered Layout Graph

For the layered layout, we used the Small Directed Network instead of League Network, because we had some issues with the implementation of the heuristic used to reverse edges in the case of League Network. We aim to solve this bug by the final delivery of the overall project.

The Figure 22 depicts the output after the node positioning step, which used barycenter heuristic. The final number of crossing were noted to be 7. It drastically went down 134 crossing before cross minimizing step to 7. We found the highest edge crossing to be 11.41° and the lowest to be 5.69° . For stress the values we got were Mean stress per pair: 0.4292 Median stress per pair: 0.4433 Max stress per pair: 0.9900 Standard deviation: 0.3454. The Figure 23 depicts the output after the node positioning step, which used median heuristic. The final number of crossing were noted to be 23. It drastically went down 103 crossing before cross minimizing step to 23. We found the highest edge crossing to be 22.52° and the lowest to be 5.68° . For stress of layout the values we got were Mean stress per pair: 1.7700 Median stress per pair: 2.0000 Max stress per pair: 5.0443 Standard deviation: 1.1856. From the above insights, we would clearly prefer the barycenter heuristic.

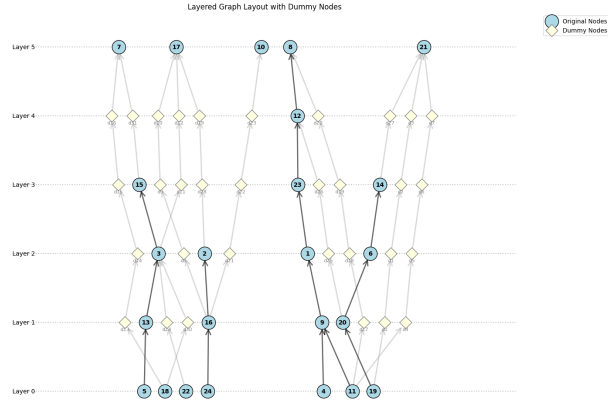


Figure 22: Small Directed Network Dataset-barycenter

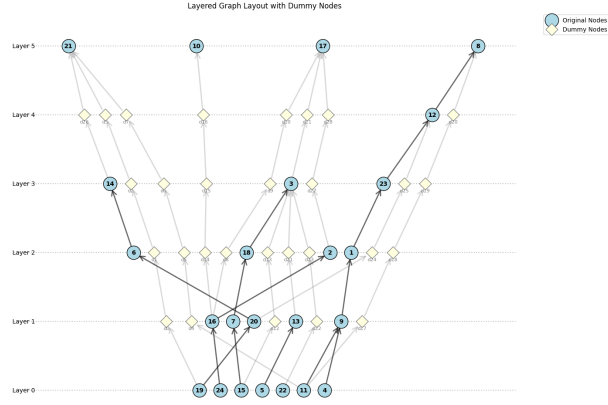


Figure 23: Small Directed Network Dataset-Median

Table 2: Comparison of Barycenter and Median Heuristics

Quality Metric	Barycenter Heuristic	Median Heuristic
Crossing Number		
Initial crossings	134	103
Final crossings	7	23
Reduction	127 (94.8%)	80 (77.7%)
Crossing Resolution (Angles)		
Highest crossing angle	11.41°	22.52°
Lowest crossing angle	5.69°	5.68°
Stress of Layout		
Mean stress per pair	0.4292	1.7700
Median stress per pair	0.4433	2.0000
Maximum stress per pair	0.9900	5.0443
Standard deviation	0.3454	1.1856

8.6 Projections for graph

For all the projections we choose a reduction to 2 components and the parameters used to create the t-SNE and ISOMAP reduced matrices of Jazz Network and Les Miserables are the following:

- t-SNE:
 - perplexity=10, angle=0.5 for Jazz Network
 - perplexity=5, angle=0.5 for Les Miserables
- ISOMAP:
 - n_neighbors = 60 for Jazz Network
 - n_neighbors = 12 for Les Miserables

The only parameter used to calculate quality metric (Trustworthiness and Continuity) is n_neighbors and it was set to 4.

Here we present all the quality metrics values for the two graphs:

Table 3: Quality Metrics Scores for Dimensionality Reduction Techniques Jazz Network

Metric	MDS	t-SNE	ISOMAP 2D
Trustworthiness	0.9535	0.7763	0.9135
Continuity	0.9401	0.7039	0.9525
Normalized Stress	0.7116	40.2336	0.7803

Table 4: Quality Metrics Scores for Dimensionality Reduction Techniques for Les Miserables

Metric	MDS	t-SNE	ISOMAP 2D
Trustworthiness	0.9618	0.8856	0.9224
Continuity	0.9248	0.9067	0.9483
Normalized Stress	0.6472	0.3023	00.6925

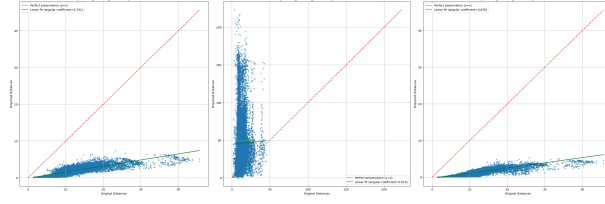


Figure 24: Shepard diagrams for MDS (left), t-SNE (center) and ISOMAP 2D (right) for Jazz Network

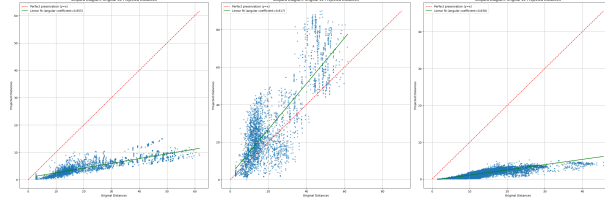


Figure 25: Shepard diagrams for MDS (left), t-SNE (center) and ISOMAP 2D (right) for Les Miserables

As we can see from the values in Tables 3 and 4 and the scatter plots in Figure 24 and Figure 25, the most effective Dimensionality Reduction Projection for these configurations seems to be the t-SNE applied to the Les Miserable graph. In fact, even though there are some projections with higher values of Trustworthiness and Continuity, this one has the lowest value of Normalized Stress and a Shepard diagram with the distribution of points close to the bisector.

8.7 Our favorite visualization technique

Overall, t-SNE turned out to be the technique we preferred for dimensionality reduction and visualization due to its easy use and ability to reveal clusters in high-dimensional data. This clarity in revealing underlying patterns, combined with practical advantages—requiring only a distance matrix as input and featuring just a few easily tunable parameters—made t-SNE the optimal choice for exploring and displaying the complex structures in our graph data.

9 Discussion

9.1 About the Project

Throughout this project, we have used 6 different types of Data Visualization Techniques. In step 4, we were not able to visualize the League Network Dataset after the cross minimization, because of the time constraints of last-minute changes to the code. For the same reason, we were not able to address all the feedbacks in step 7. Aside from that, this project systematically explored various graph visualization techniques, from basic random layouts to advanced force-directed and dimensionality reduction methods. Each step highlighted key features such as computational complexity versus visual clarity—while demonstrating how different algorithms reveal unique structural insights. The results emphasize that no single approach is universally optimal but some may be preferred for quick and easy implementation, as in the case of t-SNE. Nonetheless, the choice of the optimal visualization technique depends on the dataset, desired interpretability and computational constraints.

Throughout this project, we have used 6 different types of Data Visualization Techniques. In step 4, we were not able to visualize the League Network Dataset after the cross minimization, because of the time constraints of last-minute changes to the code. For the same reason, we were not able to address all the feedbacks in step 7. Aside from that, this project systematically explored various graph visualization techniques, from basic random layouts to advanced force-directed and dimensionality reduction methods. Each step highlighted key features such as computational complexity versus visual clarity—while demonstrating how different algorithms reveal unique structural insights. The results emphasize that no single approach is universally optimal but some may be preferred for quick and easy implementation, as in the case of t-SNE. Nonetheless, the choice of the optimal visualization technique depends on the dataset, desired interpretability and computational constraints. During the development process, large language models (LLMs) such as GitHub Copilot, Sonnet, and ChatGPT were utilized as tools to accelerate both code implementation and report writing, supporting the overall efficiency of the project workflow.

9.2 Code

The code can be found in our Github repository titled "CurryPasta" in a file titled "all_in_one_final.ipynb" in the Notebooks folder.

10 References

- [1] Thomas MJ Fruchterman and Edward M Reingold. “Graph drawing by force-directed placement”. In: *Software: Practice and experience* 21.11 (1991), pp. 1129–1164.
- [2] Danny Holten and Jarke J Van Wijk. “Force-directed edge bundling for graph visualization”. In: *Computer graphics forum*. Vol. 28. 3. Wiley Online Library. 2009, pp. 983–990.
- [3] Chun Xiao, Seokhee Hong, and Weidong Huang. “Optimizing graph layout by t-SNE perplexity estimation”. In: *International Journal of Data Science and Analytics* 15.2 (2023), pp. 159–171.
- [4] *sklearn.manifold.TSNE documentation*. scikit-learn. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>.